

T.R.
USKUDAR UNIVERSITY
FACULTY OF ENGINEERING AND NATURAL SCIENCES
DEPARTMENT OF COMPUTER ENGINEERING

GRADUATION THESIS

ANKEBUT UAV: AUTONOMOUS FOREST FIRE DETECTION AND RISK MAPPING SYSTEM USING MULTIMODAL DATA FUSION AND DEEP LEARNING

Ekrem Aydeniz

Abdullah Shikh Warak

Zehra Eraslan

Beyza Yıldırım

Thesis Advisor

Asst. Prof. Arife Aysun KARAASLAN

Date of submission: 02/07/2026

Date of defence examination: 26/06/2026

T.R.
USKUDAR UNIVERSITY
FACULTY OF ENGINEERING AND NATURAL SCIENCES

Department	: **COMPUTER ENGINEERING**
Student's No	: **210201002, 210201980, 210201028, 220201011**
Student's Name Surname	: **Ekrem Aydeniz, Abdullah Shikh Warak, Zehra Eraslan, Beyza Yıldırım**

The study named **ANKEBUT UAV: Autonomous Forest Fire Detection and Risk Mapping System Using Multimodal Data Fusion and Deep Learning** was unanimously accepted by the following jury as a Graduation Thesis in the exam held on ***26/06 /2026.***

Jury President & Advisor: **Asst. Prof. Arife Aysun KARAASLAN**
(Uskudar University)

Jury Member: **Asst. Prof. Hamza ABUNIMA**
(Uskudar University)

APPROVAL

This thesis was approved by the above jury members on **26/06/2026.**

ÖZET

ANKEBUT İHA: Çok Modlu Veri Füzyonu ve Derin Öğrenme Kullanarak Otonom Orman Yangını Tespiti ve Risk Haritalama Sistemi

Orman yangınları, Türkiye'yi ve daha geniş ölçekte Akdeniz bölgesini etkileyen en yıkıcı doğal afetler arasında yer almakta olup biyolojik çeşitlilik, ekosistemler, karbon tutma kapasitesi ve yerel topluluklar üzerinde ciddi zararlara neden olmaktadır. Yangın başlamadan önce proaktif yangın riski değerlendirmesi yapılması, etkili afet yönetimi açısından büyük önem taşımaktadır. Ancak günümüzde kullanılan birçok İHA tabanlı izleme sistemi, yangın başladıktan sonra alevleri, dumanı veya termal anomalileri tespit ederek reaktif şekilde çalışmaktadır. Ayrıca bu sistemler çoğunlukla maliyetli termal görüntüleme ekipmanlarına veya LiDAR tabanlı algılama sistemlerine bağlıdır ve bu durum geniş ölçekli uygulanabilirliklerini ve pratik kullanımlarını sınırlandırmaktadır.

Bu tez, proaktif orman yangını risk değerlendirmesi ve dinamik yangın yayılım haritalaması amacıyla tasarlanmış, tam otonom, hekzakopter tabanlı bir insansız hava aracı platformu olan ANKEBUT İHA sistemini sunmaktadır. Sistem, otonom ızgara tabanlı tarama görevleri sırasında elde edilen yüksek çözünürlüklü RGB görüntüleri kullanarak maliyet etkin ve yazılım odaklı bir yaklaşım benimsemektedir. Bu hava görüntüleri, NVIDIA Jetson Orin Nano yerleşik bilgisayarı üzerinde MobileNetV3-Large omurgasına sahip DeepLabV3 modeli kullanılarak işlenmektedir. Model, arazi örtüsünü altı sınıfa ayırmaktadır: arka plan, su, çıplak toprak, yeşil çim, kuru çim ve ağaçlar. Bu sınıflar, farklı yanıcılık seviyelerini ve yangın yayılım potansiyellerini temsil ettikleri için seçilmiştir ve sistemin daha yüksek yangın riski taşıyan bölgeleri belirlemesine olanak sağlamaktadır.

Semantik segmentasyon modeli, ImageNet üzerinde önceden eğitilmiş ağırlıklar kullanılarak, 936 eğitim görüntüsü ve 234 doğrulama görüntüsü olmak üzere toplam 1.170 manuel olarak etiketlenmiş hava görüntüsünden oluşan özel bir veri seti üzerinde eğitilmiştir. Veri seti CVAT aracı kullanılarak etiketlenmiş ve dışa aktarılan gri seviye maske değerleri, model eğitimi için uygun ardışık sınıf indekslerine dönüştürülmüştür. Eğitim, Google Colab A100 GPU üzerinde Adam optimizasyon algoritması, 1×10^{-4} öğrenme oranı ve çapraz entropi kayıp fonksiyonu kullanılarak 120 epoch boyunca gerçekleştirilmiştir. En düşük doğrulama kaybına göre seçilen en iyi model kontrol

noktası, 97. epoch'ta 0,0856 doğrulama kaybı elde etmiş ve doğrulama setinde %96,95 genel piksel doğruluğuna ulaşmıştır.

Bu tezin temel teknik katkılarından biri, rüzgâr farkındalıklı yangın yayılım riski tahmini için segmentasyon tabanlı arazi örtüsü analizinin meteorolojik verilerle bütünleştirilmesidir. Meteoroloji Genel Müdürlüğü olarak da bilinen Turkish State Meteorological Service (TSMS/MGM) tarafından sağlanan gerçek zamanlı rüzgâr yönü ve hızı verileri, segmentasyon çıktılarından elde edilen yanıcılık bilgisiyle birleştirilmektedir. Bu sayede sistem, yangının muhtemel yayılım yönünü ve göreceli risk seviyesini tahmin edebilmekte ve coğrafi referanslı risk uyarılarını gerçek zamanlı olarak yer kontrol istasyonuna iletebilmektedir.

Tam algılama-uyarı hattını güvenli saha koşullarında doğrulamak amacıyla, renk kodlu minder nesneleri üzerinde eğitilmiş özel bir YOLOv11 nesne tespit modeli kullanılarak bir vekil doğrulama protokolü uygulanmıştır. Yeşil minderler duman belirtilerini, kırmızı minderler ise aktif yangını temsil eden vekil nesneler olarak kullanılmıştır. Bu yöntem, gerçek yangın tehlikesi oluşturmadan tespit, uyarı üretimi ve yer kontrol istasyonu bildirimi süreçlerinin uçtan uca doğrulanmasını sağlamıştır. Buna ek olarak, gerçek operasyonel ortamlarda potansiyel kullanım için üretim amaçlı bir YOLOv11 yangın ve duman tespit modeli de geliştirilmiştir.

Elde edilen sonuçlar, önerilen segmentasyon ve rüzgâr verisi füzyonuna dayalı çerçevenin yangın öncesi veya erken yangın senaryolarında faydalı proaktif yangın riski bilgisi sağlayabildiğini göstermektedir. ANKEBUT İHA sistemi; otonom hava taraması, gömülü yapay zekâ, meteorolojik veri entegrasyonu ve gerçek zamanlı yer kontrol istasyonu iletişimini birleşik bir izleme platformunda başarıyla bir araya getirmektedir. Genel olarak bu tez, maliyet etkin ve yapay zekâ destekli İHA sistemlerinin, pahalı termal kameralara veya LiDAR sensörlerine bağlı kalmadan proaktif orman yangını risk değerlendirmesi için pratik bir alternatif sunabileceğini göstermektedir.

Anahtar Kelimeler: İHA, orman yangını tespiti, semantik segmentasyon, DeepLabV3, MobileNetV3-Large, bitki örtüsü sınıflandırması, yangın riski haritalaması, rüzgâr verisi entegrasyonu, TSMS/MGM, YOLOv11, vekil doğrulama, otonom hava sistemleri.

ABSTRACT

ANKEBUT UAV: Autonomous Forest Fire Detection and Risk Mapping System Using Multimodal Data Fusion and Deep Learning

Forest fires are among the most destructive natural disasters affecting Turkey and the broader Mediterranean region, causing severe damage to biodiversity, ecosystems, carbon sequestration capacity, and local communities. Proactive fire risk assessment before ignition is essential for effective disaster management; however, many current UAV-based monitoring systems operate reactively by detecting flames, smoke, or thermal anomalies only after a fire has already started. In addition, such systems often depend on costly thermal imaging equipment or LiDAR-based sensing, which limits their scalability and practical deployment.

This thesis presents the ANKEBUT UAV system, a fully autonomous hexacopter-based unmanned aerial vehicle platform designed for proactive forest fire risk assessment and dynamic fire spread mapping. The system adopts a cost-effective, software-driven approach by using high-resolution RGB imagery collected during autonomous grid-based scanning missions. These aerial images are processed on an NVIDIA Jetson Orin Nano onboard computer using a DeepLabV3 model with a MobileNetV3-Large backbone. The model classifies land cover into six categories: background, water, bare soil, green grass, dry grass, and trees. These classes were selected because they represent different levels of combustibility and fire spread potential, enabling the system to identify areas with higher wildfire risk.

The semantic segmentation model was trained using ImageNet-pretrained weights on a custom dataset consisting of 1,170 manually annotated aerial images, including 936 training images and 234 validation images. The dataset was annotated using the CVAT tool, and the exported grayscale mask values were post-processed into sequential class

indices suitable for model training. Training was performed for 120 epochs using the Adam optimizer with a learning rate of 1×10^{-4} and cross-entropy loss on a Google Colab A100 GPU. The best model checkpoint, selected according to the minimum validation loss of 0.0856 at epoch 97, achieved an overall pixel accuracy of 96.85% on the validation set.

A key technical contribution of this thesis is the integration of segmentation-based land cover analysis with meteorological data for wind-aware fire spread risk estimation. Real-time wind direction and speed data obtained from the Turkish State Meteorological Service, also known as Meteoroloji Genel Müdürlüğü (TSMS/MGM), are fused with segmentation-derived combustibility information. This enables the system to estimate the probable direction and relative risk of fire spread and to generate geo-referenced risk alerts that are transmitted to the ground control station in real time.

To validate the complete detection-to-alert pipeline under safe field conditions, a proxy detection protocol was implemented using a dedicated YOLOv11 object detection model trained on color-coded pillow objects. Green pillows were used as proxies for smoke signatures, while red pillows were used as proxies for active fire. This method enabled end-to-end validation of detection, alert generation, and ground control station notification without creating real fire hazards. In addition, a production YOLOv11 fire and smoke detection model was developed for potential deployment in real operational environments.

The results demonstrate that the proposed segmentation-and-wind-fusion framework can provide useful proactive fire risk information before or during early fire scenarios. The ANKEBUT UAV system successfully combines autonomous aerial scanning, embedded artificial intelligence, meteorological data integration, and real-time ground station communication into a unified monitoring platform. Overall, this thesis shows that cost-effective, AI-supported UAV systems can offer a practical alternative for proactive wildfire risk assessment without relying on expensive thermal cameras or LiDAR sensors.

Keywords: UAV, forest fire detection, semantic segmentation, DeepLabV3, MobileNetV3-Large, vegetation classification, fire risk mapping, wind data integration, TSMS/MGM, YOLOv11, proxy validation, autonomous aerial systems.

THANKS TO

We wish to extend our deepest gratitude to our thesis advisor, Asst. Prof. Arife Aysun Karaaslan, whose continuous guidance and academic insight made this research possible. Her supervision was essential in shaping the direction of this work, and we are highly appreciative of the time and knowledge she invested in our graduation project.

FORM OF DECLARATION

Herewith we declare, that we obtained all the information and documents in this study within the framework of academic rules, presented all visual, auditory, and written information and results in accordance with scientific ethics, did not falsify the data we used, referred to the sources we used in accordance with scientific norms, that our thesis was original except in the cases cited, produced by us and written in accordance with the guidelines of Uskudar University.

CONTENTS

ÖZET ..... **i**
ABSTRACT ..... **iii**
THANKS TO ..... **v**
FORM OF DECLARATION ..... **vi**
CONTENTS ..... **vii**
INDEX OF TABLES ..... **x**
INDEX OF FIGURES ..... **xi**
INDEX OF IMAGERY AND ABBREVIATIONS ..... **xii**
1. INTRODUCTION **1**
1.1. Purpose of the Thesis 2
1.2. Background 2
1.3. Hypothesis 3
1.4. Objective 4
2. MATERIALS AND METHODS **5**
2.1. Chapter Objectives 5
2.2. Research Methodology 5
2.3. System Architecture 6
2.3.1. General System Structure 7
2.3.2. Airframe and Mechanical Layout 7
2.3.3. Flight Control Subsystem 8
2.3.4. Navigation and Autonomous Mission Logic 10
2.3.5. Companion Computer Subsystem 12
2.3.6. Imaging Subsystem 13
2.3.7. Perception and Risk Assessment Pipeline 14
2.3.8. Communication and Telemetry Subsystem 15
2.3.9. Power Distribution Subsystem 15
2.3.10. Architectural Rationale 16
2.4. UAV Airframe and Propulsion System 16

2.5. Software Architecture Overview 18
2.5.1. Development and Execution Environment 19
2.6. Land Cover Segmentation: Model Design and Training 20
2.6.1. Architecture: DeepLabV3 with MobileNetV3-Large Encoder 20
2.6.2. Dataset Composition, Annotation and Augmentation 21
2.6.2.1. Dataset Preparation and Cropping 22
2.6.2.2. Manual Annotation Using CVAT 22
2.6.2.3. Data Augmentation 23
2.6.2.4. Mask Preprocessing and Dataset Split 25
2.6.3. Training Configuration 26
2.6.4. Segmentation Classes and Combustibility Mapping 27
2.7. Wind Data Integration and Risk Warning System 28
2.7.1. TSMS / MGM Wind Data Retrieval 28
2.7.2. Risk Fusion Computation 29
2.8. Fire and Smoke Detection: YOLOv11 Models: 30
2.8.1. Purpose of the Detection Module 31
2.8.2. Production Fire and Smoke Detection Model 31
2.8.3. Dataset Balance and Annotation Refinement for the Production Model 34
2.8.4. Proxy Pillow Detection Model for Safe Validation 35
2.8.5. Proxy YOLOv11 Training Configuration 36
2.8.6. Proxy Model Results and Validation 37
2.8.7. Integration with the ANKEBUT Alert Pipeline 38
2.9. Ground Control Station Software 39
2.9.1. Architectural Design Philosophy 39
2.9.2. Mission Planner Integration 40
2.9.3. Real-Time Video Reception Pipeline 41
2.9.4. Wireless Communication Infrastructure 42

2.9.5. MAVLink Telemetry Reception 44
2.9.6. GCS Software Module Summary 45
2.9.7. Communication Performance and Link Reliability 46
2.9.8. Operator Interface and Usability Considerations 46
3. RESULTS AND DISCUSSION 48
3.1. System Implementation Overview 48
3.2. UAV Hardware Integration and Mission Performance 49
3.3. Land Cover Segmentation Results 50
3.4. YOLOv11 Proxy Detection Results 53
3.5. Wind-Aware Risk Assessment Output 55
3.6. Ground Control Station Integration Results 56
3.7. Autonomous Mission and End-to-End Validation 58
3.8. Discussion 61
4. CONCLUSION AND FUTURE WORK 63
4.1. Conclusion 63
4.2. Main Contributions 64
4.3. Limitations 65
4.4. Future Work 67
REFERENCES 69
APPENDIX 70
Appx. A. COST ANALYSIS AND BUDGET TABLE 70

INDEX OF TABLES

Page

Table 2.1 Main airframe and propulsion specifications of the ANKEBUT UAV..... 18
Table 2.2 Training configuration and validation summary of the DeepLabV3-MobileNetV3-Large segmentation model. 27
Table 2.3 Semantic segmentation classes and combustibility coefficients used in the ANKEBUT risk assessment pipeline. 28
Table 2.4 Production fire/smoke dataset balance before and after reducing smoke-heavy samples. 34
Table 2.5 Training configuration of the proxy YOLOv11 detection model. 37
Table 2.6 GCS video reception pipeline configuration parameters. 42
Table 2.7 Jetson Orin Nano wireless access point configuration. 44
Table 2.8 GCS software modules and their roles. 45
Table 3.1 UAV hardware integration and mission performance indicators observed during system testing. 50
Table 3.2 Selected training and validation loss values of the DeepLabV3-MobileNetV3-Large segmentation model. 51
Table 3.3 Final validation performance of the DeepLabV3-MobileNetV3-Large segmentation model. 51
Table 3.4 Validation performance of the proxy YOLOv11 detection model. 54
Table 3.5 Functional validation summary of the wind-aware risk assessment stage. 56

Table A 1 Cost analysis and budget breakdown of the ANKEBUT UAV project. 70

INDEX OF FIGURES

Page

Figure 2.1 CAD model of the ANKEBUT S550 hexacopter UAV platform 8

Figure 2.2 Hardware connection architecture of the ANKEBUT UAV system..... 9

Figure 2.3 Physical integration of the ANKEBUT UAV hardware 10

Figure 2.4 NVIDIA Jetson Orin Nano companion computer..... 12

Figure 2.5 Perception and risk assessment pipeline of the ANKEBUT UAV system from RGB camera input to ground station alert generation..... 12

Figure 2.6 Manual land cover annotation process in CVAT 23

Figure 2.7 Augmented aerial image after applying visual transformation operations.... 24

Figure 2.8 Original aerial image before data augmentation. 24

Figure 2.9 Ground control station software and communication architecture: PySide6 application layout with embedded Mission Planner, live video panel, and telemetry panel. 44

Figure 2.10 Ground-truth semantic segmentation mask..... 26

Figure 2.11 Revised production fire/smoke dataset label distribution 33

Figure 2.12 Initial production fire/smoke dataset label distribution before balancing. .. 33

Figure 2.13 Bounding box annotation example..... 36

Figure 3. 1 Qualitative validation result 53

Figure 3. 2 DeepLabV3-MobileNetV3-Large segmentation model..... 53

Figure 3. 3 Proxy YOLOv11 detection model. 54

Figure 3. 4 Qualitative validation result of the proxy YOLOv11 detection model 55

Figure 3.5 Ground Control Station software interface showing embedded Mission Planner, live video display, and telemetry summary panel. 57

Figure 3.6 GCS interface during proxy fire detection, showing the live detection overlay, fire alert status, UAV telemetry, and climbing response. 63

Figure 3.7 GCS interface during the analysis stage after proxy fire detection, showing segmentation-based visual processing, fire alert status, telemetry, and Mission Planner monitoring..... 63

INDEX OF IMAGERY AND ABBREVIATIONS

AP: Access Point

API: Application Programming Interface

COTS: Commercial Off-The-Shelf

CVAT: Computer Vision Annotation Tool

DHCP: Dynamic Host Configuration Protocol

ESC: Electronic Speed Controller

FPS: Frames Per Second

GCS: Ground Control Station

GNSS: Global Navigation Satellite System

GPS: Global Positioning System

GPU: Graphics Processing Unit

IMU: Inertial Measurement Unit

IoU: Intersection over Union

İHA: İnsansız Hava Aracı (Turkish for UAV)

LWIR: Long-Wave Infrared

MAVLink: Micro Air Vehicle Communication Protocol

MGM: Meteoroloji Genel Müdürlüğü (Turkish State Meteorological Service)

mIoU: Mean Intersection over Union

MPEG-TS: MPEG Transport Stream

MTOW: Maximum Take-Off Weight

RGB: Red, Green, Blue (visible-light camera)

ROI: Region of Interest

ROS2: Robot Operating System 2

RTL: Return-to-Launch

SSID: Service Set Identifier

TSMS: Turkish State Meteorological Service

UAV: Unmanned Aerial Vehicle

UDP: User Datagram Protocol

Wi-Fi: Wireless Fidelity (IEEE 802.11)

WPA2: Wi-Fi Protected Access 2

YOLO: You Only Look Once (real-time object detection framework)

1. INTRODUCTION

Forest fires are among the most destructive and recurring natural hazards affecting Türkiye and the Mediterranean region. According to wildfire monitoring literature, forest fire events create long-term consequences for biodiversity, soil integrity, carbon sequestration, and regional economies **[10]**. The increasing frequency and intensity of fire events, driven by accelerating climate change, extended drought periods, and rising ambient temperatures, has created an urgent need for more advanced, proactive, and cost-scalable fire risk monitoring technologies.

Conventional fire detection infrastructure, including fixed watchtowers, satellite remote sensing, and manned aerial patrols, suffers from well-documented limitations: limited spatial coverage, delayed detection, high operational costs, and an inherently reactive character **[10]**. These systems predominantly identify fires only after ignition and visible flame or smoke formation have already occurred. Unmanned aerial vehicles (UAVs) have emerged as compelling platforms for environmental monitoring, offering flexible deployment, real-time data acquisition, and cost-effective area coverage compared to manned alternatives **[6]**.

Existing UAV-based fire detection systems share the reactive paradigm of their ground-based counterparts; most are designed to detect active flames or thermal anomalies using Long-Wave Infrared (LWIR) thermal cameras, with detection occurring only after a fire is already underway **[6]**. Such systems lack the capability to characterize pre-ignition risk based on vegetation combustibility, fuel load density, and meteorological conditions. Moreover, the high cost of LWIR thermal cameras limits the large-scale use of UAV systems over wide areas.

This thesis addresses these limitations through the ANKEBUT UAV system, which takes a fundamentally different approach; rather than detecting fires reactively, the system performs proactive risk assessment by classifying land cover combustibility from RGB imagery using a trained DeepLabV3 segmentation model **[2]** and fusing the resulting risk estimate with real-time meteorological wind data from the Turkish State Meteorological Service (TSMS/MGM) **[9]**. This allows the system to generate directional fire spread risk predictions before any ignition event occurs.

1.1. Purpose of the Thesis

The primary purpose of this thesis is to design, implement, and evaluate the ANKEBUT UAV system as a fully autonomous aerial platform for proactive forest fire risk assessment and dynamic fire spread mapping. In this context, the thesis aims to develop a fully autonomous hexacopter UAV capable of executing pre-planned grid-based forest scanning missions with minimal human intervention; to design, train, and evaluate a DeepLabV3 model with a MobileNetV3-Large encoder backbone for six-class aerial land cover classification using a custom annotated dataset **[2][4]**, and to develop a wind-integrated fire spread risk warning system that retrieves real-time wind direction and speed data from the TSMS/MGM API **[9]** and fuses this information with segmentation-derived combustibility maps to generate geo-referenced and directionally aware risk alerts. In addition, the thesis aims to design and implement a proxy detection protocol using a YOLOv11 model trained on color-coded pillow surrogates **[5]**, where green objects represent smoke and red objects represent fire, to enable safe end-to-end validation of the full detection-to-alert pipeline without real fire hazards. Finally, the thesis evaluates the overall system performance through controlled field tests and quantitative metrics, while assessing the practical feasibility of the proposed cost-effective approach as a scalable alternative to UAV systems that depend on expensive thermal cameras or LiDAR sensors.

1.2. Background

Research in UAV-based wildfire monitoring has expanded substantially over the past decade **[10]**. Early systems primarily relied on Long-Wave Infrared (LWIR) thermal cameras to detect surface temperature anomalies that indicate active fires **[6]**. While effective for detecting ongoing combustion, thermal-based systems provide limited ability to characterize fire spread potential or to generate proactive risk maps from pre-ignition land conditions.

The introduction of deep learning-based object detection, particularly YOLO-family architectures, expanded the scope of RGB-camera-based fire detection **[8]**. These systems achieve promising detection accuracy in controlled settings; however, RGB-only approaches remain susceptible to false positives caused by optical factors such as sun glare, reflective soil, and shadows, which can reduce operational reliability in complex natural environments.

Semantic segmentation has been increasingly applied in remote sensing for land cover classification, enabling detailed characterization of vegetation types from aerial imagery. The DeepLabV3 architecture family uses atrous convolution layers to capture wider image context and improve multi-scale feature representation **[2][3]**. The MobileNetV3-Large encoder variant offers a favorable balance between accuracy and processing speed, making it suitable for real-time embedded inference on resource-constrained mission computers such as the NVIDIA Jetson Orin Nano **[4]**.

The integration of meteorological data with UAV-based sensing for fire risk modeling represents an emerging direction in the literature. Wind conditions are well established as major factors affecting fire spread rate and direction in fire behavior models **[7][1]**. Incorporating real-time wind vector data from national meteorological services into an aerial monitoring pipeline offers a cost-effective route to directionally aware risk prediction without onboard wind sensors **[9]**. The ANKEBUT UAV system builds upon this direction by combining DeepLabV3-MobileNetV3-Large segmentation with TSMS/MGM wind data within a real-time embedded UAV architecture.

1.3. Hypothesis

The central hypothesis of this thesis is that semantic vegetation segmentation of aerial RGB imagery using a DeepLabV3-MobileNetV3-Large model, when integrated with real-time meteorological wind data within an autonomous UAV platform, can provide useful and actionable proactive fire spread risk assessments. It is expected that the fusion of segmentation-derived combustibility information and wind direction data will improve the spatial specificity and directional usefulness of the generated risk warnings compared to using visual land cover information alone.

It is further hypothesized that the trained DeepLabV3-MobileNetV3-Large model will achieve sufficient segmentation accuracy on the five land cover classes, together with the background class, to support reliable combustibility classification under real-world aerial imaging conditions. Due to the lightweight MobileNetV3-Large backbone, the model is also expected to provide adequate inference performance for real-time onboard deployment on the NVIDIA Jetson Orin Nano without requiring additional model compression.

In addition, it is hypothesized that real-time wind direction and speed data obtained from TSMS/MGM can support directionally aware fire spread risk estimation and provide operationally useful warning information. Finally, a YOLOv11 model trained on color-coded pillow surrogates is expected to serve as a practical proxy for validating the full fire and smoke detection-to-alert pipeline under controlled field conditions, without the safety hazards or regulatory limitations associated with live fire testing.

1.4. Objective

The main objectives of this thesis are to develop a fully autonomous UAV platform capable of executing grid-based forest scanning missions with minimal human intervention; to design and train a DeepLabV3-based semantic segmentation model for aerial land cover classification, including water, bare soil, green grass, dry grass, and trees, in addition to the background class; and to evaluate the suitability of the selected DeepLabV3-MobileNetV3-Large architecture for real-time embedded inference. In addition, this thesis aims to integrate real-time wind direction and speed data obtained from the *TSMS/MGM* into the risk assessment pipeline, implement a wind-aware fire spread risk warning system that generates geo-referenced alerts based on the fusion of vegetation segmentation and meteorological data, evaluate the system's performance through controlled flight tests, and validate the accuracy of the segmentation outputs and risk map generation. Finally, the thesis aims to demonstrate the feasibility of a cost-effective, AI-powered proactive fire risk platform as an alternative to systems that depend on expensive thermal cameras or LiDAR sensors.

2. MATERIALS AND METHODS

2.1. Chapter Objectives

The objective of this chapter is to present, in technical detail, the design decisions, hardware components, software architecture, machine learning models, and integration methodology that together constitute the ANKEBUT UAV system. The chapter aims to provide sufficient information for the reproducibility of the core system components and to document the engineering rationale behind each major design choice. Specifically, this chapter describes the airframe, propulsion, avionics, and sensing hardware components selected for the ANKEBUT UAV system and justifies each selection in terms of cost-effectiveness, performance suitability, and integration compatibility. It also details the design and training methodology of the DeepLabV3-MobileNetV3-Large land cover segmentation model, including dataset composition, annotation procedure, preprocessing, hyperparameter selection, and training configuration. Furthermore, the chapter explains the wind data integration pipeline and the risk fusion computation that converts segmentation outputs and TSMS/MGM meteorological data into geo-referenced, directionally aware fire spread risk warnings. In addition, it documents both the production YOLOv11 fire and smoke detection model and the proxy YOLOv11 validation model, including their respective training configurations, datasets, and the rationale behind the proxy detection methodology. Finally, the chapter describes the autonomous mission execution logic, safety mechanisms, and ground control station communication architecture.

2.2. Research Methodology

The research methodology adopted in this thesis follows an iterative, hardware-in-the-loop development and validation cycle. The overall approach is summarized in five sequential phases:

- *Phase 1:* System Design: Requirement analysis: hardware selection, and software architecture definition based on the operational constraints of proactive UAV forest fire risk assessment (cost-effectiveness, real-time inference, regulatory safety compliance).
- *Phase 2:* Dataset Acquisition and Annotation: Aerial RGB imagery was collected via UAV flight sorties over diverse Turkish terrain types (forest, scrubland, dry

grass, open soil, water bodies). Images were manually annotated using the CVAT tool with six-class land cover labels and exported as grayscale masks with class-specific pixel values.

- *Phase 3:* Model Training and Evaluation: The DeepLabV3-MobileNetV3-Large segmentation model was trained from COCO-pretrained weights on the custom annotated dataset using an Adam optimizer with cross-entropy loss. YOLOv11-based detection models (production and proxy) were independently trained using the Ultralytics framework on Google Colab. Models were evaluated on held-out validation sets using quantitative performance metrics.
- *Phase 4:* System Integration and Software Development: The trained model weights were integrated into the software stack deployed on the NVIDIA Jetson Orin Nano mission computer. The risk fusion module, alert generator, MAVLink communication module, UDP video transmission pipeline, ground control station communication components, and the ground-station wind data client were developed and tested separately before full-stack integration.
- *Phase 5:* Field Validation: End-to-end system validation was conducted through controlled field trials employing the proxy YOLOv11 detection model (green pillow = smoke, red pillow = fire). All system components were exercised under real flight conditions, and detection-to-alert latency, risk map generation, and GCS notification were measured against defined performance targets.

This methodology ensures that each system layer is independently verifiable before full-stack integration, reducing the risk of compounded failure modes and providing clear traceability between design decisions and measured outcomes.

2.3. System Architecture

The ANKEBUT UAV system is designed around a modular architecture that separates flight control, onboard artificial intelligence processing, sensing, communication, and ground station monitoring. This modular structure allows each subsystem to operate with clear responsibility while remaining integrated within the completely autonomous UAV platform. The main purpose of this architecture is to ensure stable autonomous flight, real-time image processing, fire risk assessment, and reliable communication with the ground control station.

2.3.1. General System Structure

The ANKEBUT UAV system consists of two main operational layers: the flight control and autonomy layer, and the perception and risk assessment layer.

- The flight control and autonomy layer is responsible for:
- Safe flight operation.
- Stabilization.
- Waypoint navigation.
- Grid-based mission execution.
- Telemetry.
- Fail-safe actions.

This layer is managed by the Cube Orange flight controller running ArduPilot firmware. The perception and risk assessment layer is responsible for image acquisition, artificial intelligence inference, wind data integration, risk estimation and alert generation. This layer is managed by the NVIDIA Jetson Orin Nano companion computer using a Python-based software pipeline.

This separation was selected because flight control tasks are time-critical and must always remain stable, while artificial intelligence processing requires higher computational resources. By separating these two responsibilities, the UAV can maintain safe autonomous flight while performing real-time fire risk analysis.

2.3.2. Airframe and Mechanical Layout

The mechanical structure of the ANKEBUT UAV is based on an S550 hexacopter frame. The hexacopter configuration was selected because it provides stable hovering, balanced payload distribution and better tolerance to propulsion-related disturbances compared to a quadrotor configuration. Before physical integration, the general mechanical layout of the UAV was designed and visualized using Fusion 360, as shown in Figure 2.1. This CAD design was used to examine the frame geometry, propeller spacing, landing gear clearance and the placement of major onboard components such as the flight controller, companion computer (Jetson Orin Nano), camera, battery and communication modules.



Figure 2.1 CAD model of the ANKEBUT S550 hexacopter UAV platform showing the main airframe and onboard component placement.

As shown in Figure 2.1, the S550 frame provides sufficient space for distributing the propulsion units symmetrically around the central body while leaving the middle section available for avionics and payload integration. This arrangement supports stable center-of-gravity distribution and reduces the risk of interference between rotating propellers and onboard components. The CAD layout also helped verify that the lower part of the frame provides enough clearance for the camera and landing gear, which is important for safe take-off, landing, and image acquisition during flight.

2.3.3. Flight Control Subsystem

The primary flight control unit of the ANKEBUT UAV is the Cube Orange flight controller. This unit is responsible for stabilizing the UAV during flight, processing navigation inputs, generating motor control signals, and managing autonomous flight modes. The Cube Orange was selected because it supports multirotor UAV platforms, provides reliable sensor-based stabilization, and is compatible with the ArduPilot firmware.

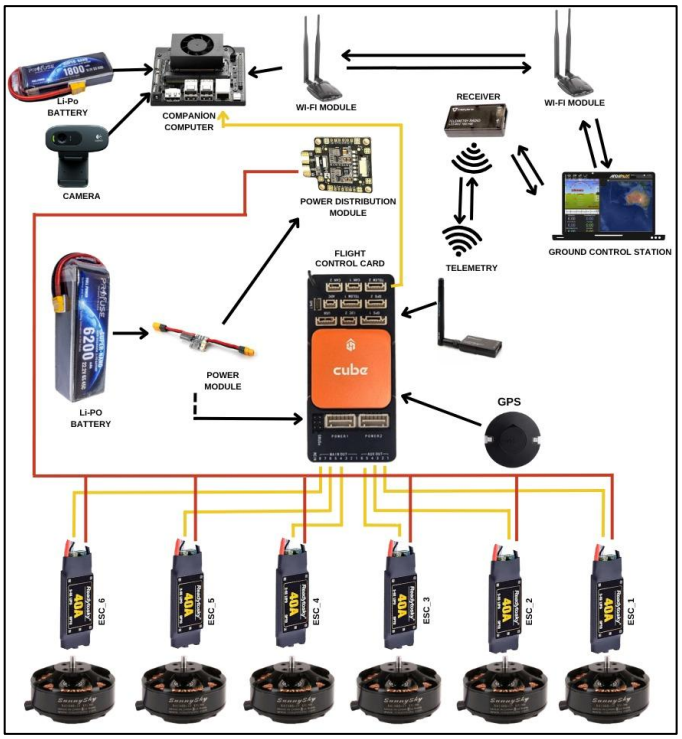


Figure 2.2 Hardware connection architecture of the ANKEBUT UAV system showing the main power, flight control, companion computer, camera, telemetry, GPS, and ground station communication links.

As shown in Figure 2.2, the Cube Orange is positioned as the central flight-control unit connected to the ESCs, motors, GPS module, telemetry unit, and companion computer. This connection structure allows the flight controller to manage low-level flight behavior while exchanging mission-related data with the Jetson Orin Nano through MAVLink communication.

The main responsibilities of the flight control subsystem are:

• Low-level flight stabilization and attitude control.

• Motor command generation through the ESCs.

• GPS-guided waypoint navigation.

• Grid-based forest scanning mission execution.

• Battery monitoring and flight status supervision.

• Telemetry transmission to the ground control station.

• Fail-safe actions such as Return-to-Launch (RTL) and emergency landing.

Since this subsystem is responsible for flight safety, it operates independently from the onboard artificial intelligence processing pipeline. This ensures that possible delays in image processing or risk estimation do not affect the stability of the UAV.



Figure 2.3 Physical integration of the ANKEBUT UAV hardware showing the Jetson Orin Nano, Cube Orange flight controller, onboard electronics, wiring, and central component placement on the S550 frame.

2.3.4. Navigation and Autonomous Mission Logic

The navigation subsystem of the ANKEBUT UAV provides the position, altitude, and heading information required for autonomous mission execution. A GNSS/GPS module is used to support waypoint navigation, grid-based scanning, Return-to-Launch (RTL) behavior, and geo-referencing of detected risk areas. The position data obtained from this subsystem are used by both the Cube Orange flight controller and the Jetson Orin Nano companion computer. The Cube Orange uses this information for autonomous navigation and flight control, while the Jetson Orin Nano uses it to associate detected fire risk regions with real-world geographic coordinates.

The ANKEBUT UAV executes pre-planned grid-based scanning missions uploaded to the Cube Orange through the ground control station. This grid-based mission structure enables systematic area coverage and reduces the probability of leaving unscanned regions within the target environment. During flight, the Cube Orange is responsible for maintaining stable autonomous navigation, following the planned waypoints, controlling altitude and heading, and managing flight modes. At the same time, the Jetson Orin Nano processes the RGB camera stream, performs land cover segmentation and YOLOv11-based detection, and generates risk assessment outputs, while wind information for the flagged location is retrieved on the ground control station side. The autonomous mission begins with an automated take-off to a safe altitude defined within the uploaded mission,

after which the UAV proceeds through the planned scanning waypoints. When a fire- or smoke-related detection is confirmed over successive frames, the companion computer temporarily switches the flight controller to a guided hold above the detected location so that land cover segmentation and wind-aware risk analysis can be performed, and then returns control to the autonomous mission so that grid scanning resumes; a short cooldown period prevents the same target from repeatedly re-triggering this behaviour.

When a geo-referenced fire risk alert is generated, the system transmits the warning information to the ground control station. The alert includes the estimated risk location, risk level, and probable spread direction. This allows the operator to monitor the flagged area more closely and, if necessary, update the mission plan for further inspection. In this way, the Cube Orange remains responsible for safe flight execution, while the Jetson Orin Nano provides high-level perception and risk awareness during the mission.

Several safety mechanisms are integrated into the mission logic to protect the UAV during autonomous operation. Battery voltage is monitored continuously, and Return-to-Launch (RTL) can be triggered when the battery reaches a low threshold. If the battery reaches a critical level, landing behavior can be initiated to prevent unsafe flight continuation. Communication loss beyond a defined timeout is handled by the ArduPilot failsafe system, which can automatically trigger RTL according to the configured safety parameters.

Throughout the mission, flight data, telemetry information, visual outputs, and generated alerts are transmitted to the ground control station for operator monitoring. This integrated navigation and mission logic allows the ANKEBUT UAV to perform autonomous grid-based scanning while maintaining safety, operator supervision, and real-time fire risk awareness.

The main responsibilities of the navigation and autonomous mission subsystem are:

- Providing UAV position, altitude, and heading information.
- Supporting autonomous waypoint navigation.
- Enabling grid-based mission execution.
- Supporting Return-to-Launch (RTL) behavior.
- Providing location data for geo-referenced fire risk alerts.
- Supporting safe mission execution through battery and communication failsafe behaviors.

- Enabling operator supervision through real-time telemetry and alert transmission.

2.3.5. Companion Computer Subsystem

The NVIDIA Jetson Orin Nano is used as the onboard companion computer of the ANKEBUT UAV. It was selected because it provides embedded GPU computing capability for real-time deep learning inference while remaining compact and suitable for UAV integration. The Jetson Orin Nano includes an NVIDIA Ampere GPU with CUDA cores and Tensor Cores, which allows deep learning models to run more efficiently than on a standard CPU-only embedded computer. This is important for the ANKEBUT system because semantic segmentation and object detection must be performed during flight with low processing delay.

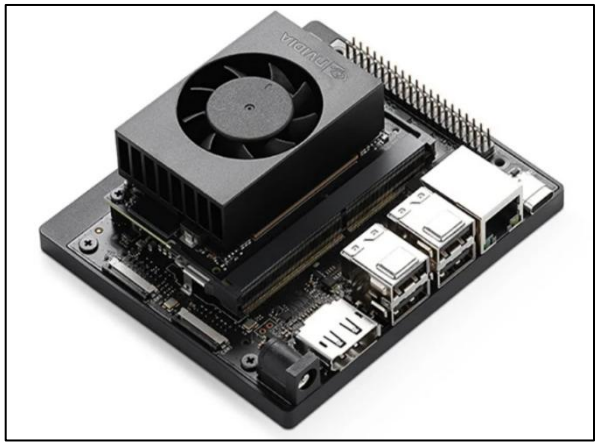


Figure 2.4 NVIDIA Jetson Orin Nano companion computer used for onboard AI inference, camera processing, telemetry handling, and risk assessment in the ANKEBUT UAV system.

In the ANKEBUT UAV, Jetson Orin Nano is responsible for processing the RGB camera stream, running the DeepLabV3-MobileNetV3-Large semantic segmentation model, running the YOLOv11 detection model, and generating fire spread risk alerts, while the wind data for the flagged location is retrieved separately by the ground control station. The use of CUDA acceleration allows the system to benefit from parallel GPU processing during AI inference, especially for image-based tasks such as segmentation and object detection.

In the hardware architecture shown in Figure 2.2, the Jetson Orin Nano is connected to the camera for image acquisition and to the Cube Orange flight controller for telemetry exchange. In the software pipeline shown in Figure 2.5, the Jetson Orin Nano acts as the central processing unit where frame acquisition, AI inference, risk fusion, fire spread

estimation, and alert generation are performed. Figure 2.4 shows the NVIDIA Jetson Orin Nano companion computer used as the onboard AI processing unit in the ANKEBUT UAV system.

The main responsibilities of the companion computer subsystem are:

- Receiving live RGB frames from the onboard camera.
- Running the land cover segmentation model using GPU acceleration.
- Running the fire/smoke or proxy detection model.
- Receiving telemetry data from the Cube Orange through MAVLink.
- Providing the geo-referenced location that the ground control station uses to retrieve wind data.
- Combining segmentation, detection, GPS, and wind data.
- Generating geo-referenced and directionally aware of fire risk alerts.

2.3.6. Imaging Subsystem

The imaging subsystem consists of an RGB camera connected to the Jetson Orin Nano. The camera provides the visual input required for land cover segmentation, fire/smoke-related detection, and live video monitoring. RGB imagery was selected as the primary sensing input because it supports a cost-effective, software-driven approach without requiring expensive thermal cameras or LiDAR sensors.

As shown in Figure 2.5, the RGB camera input is the first stage of the perception pipeline. The captured frames are sent to the frame acquisition stage, where they are prepared for artificial intelligence inference. These frames are then used by the DeepLabV3-MobileNetV3-Large model for land cover segmentation and by the YOLOv11 model for fire/smoke-related detection or proxy target detection during testing.

The main responsibilities of the imaging subsystem are:

• Capturing aerial RGB images during flight.

• Providing input frames for semantic segmentation.

• Providing input frames for YOLOv11-based object detection.

• Supporting live video transmission to the ground control station.

• Enabling visual monitoring of the scanned forest area.

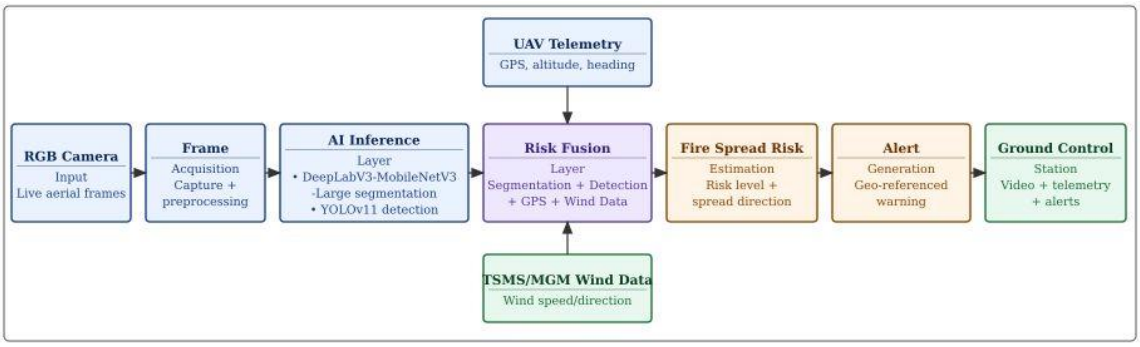


Figure 2.5 Perception and risk assessment pipeline of the ANKEBUT UAV system from RGB camera input to ground station alert generation.

2.3.7. Perception and Risk Assessment Pipeline

The perception and risk assessment pipeline runs on the Jetson Orin Nano and is illustrated in Figure 2.5. The pipeline begins with RGB camera input and frame acquisition. After the image frames are captured, they are processed by the AI inference layer. This layer includes the DeepLabV3-MobileNetV3-Large semantic segmentation model and the YOLOv11 detection model.

The DeepLabV3-MobileNetV3-Large model classifies land cover regions such as water, bare soil, green grass, dry grass and trees, in addition to the background class. These outputs are used to estimate the spatial distribution of combustible and non-combustible areas. In parallel, the YOLOv11 detection model is used for fire and smoke-related detection tasks. During controlled field validation, color-coded pillow objects are used as proxy targets, where green pillows represent smoke and red pillows represent active fire. This allows the complete detection-to-alert pipeline to be tested safely without real fire hazards.

As shown in the risk fusion layer of Figure 2.5, the system combines several data sources before generating the final warning. These inputs include:

• Land cover segmentation results.

• YOLOv11 detection outputs.

• UAV GPS position and heading.

• Real-time wind direction and speed data from TSMS/MGM.

Using these inputs, the system estimates the fire spread risk and produces a geo-referenced warning that includes the risk location, risk level, and probable spread

direction. The final stages of Figure 2.5 show how the generated warning is transmitted to the ground control station for operator monitoring.

2.3.8. Communication and Telemetry Subsystem

The communication and telemetry subsystem provides data exchange between the flight controller, companion computer and ground control station. Communication between the Cube Orange flight controller and the Jetson Orin Nano is established through a USB serial connection using the MAVLink protocol. Through this link, the Jetson Orin Nano receives telemetry data such as UAV position, altitude, heading, battery status and mission state. These data are required to associate image-based risk outputs with real-world geographic coordinates.

Communication with the ground control station is provided through the wireless communication link. This link allows the UAV to transmit mission-related data, live video, telemetry information and the generated fire risk alerts to the operator. The ground control station is discussed here only as part of the overall system communication architecture, while its detailed software design, Mission Planner integration, video reception pipeline, telemetry display and user interface structure are explained separately later in Section 2.9.

The main responsibilities of the communication and telemetry subsystem are:

• Providing MAVLink communication between the Cube Orange and Jetson Orin Nano through USB serial communication.

• Transmitting flight telemetry to the ground control station.

• Sending live video to the ground control station.

• Sending geo-referenced fire risk alerts.

• Supporting operator supervision and intervention when required.

2.3.9. Power Distribution Subsystem

The power distribution subsystem supplies electrical power to both the propulsion system and the onboard electronic components of the ANKEBUT UAV. The main propulsion system is powered by a 6S lithium polymer (LiPo) battery, which supplies power to the ESCs and motors through the power distribution module. This main battery is responsible for providing the high-current output required for flight.

In addition to the main propulsion battery, the NVIDIA Jetson Orin Nano companion computer is powered by a separate 3S 1800 mAh battery. This separation was preferred to reduce the effect of voltage drops and current fluctuations caused by the propulsion system on the onboard artificial intelligence computer. Since the motors and ESCs can draw high current during take-off, maneuvering and wind correction, powering the Jetson from an independent battery helps improve the stability and reliability of the perception and risk assessment pipeline.

As shown in Figure 2.2, the main power distribution module acts as the connection point between the 6S battery and the propulsion-related electrical components of the UAV. Regulated power lines are used where necessary for sensitive electronic components such as the flight controller, camera, and communication modules. Proper power distribution is important because the propulsion system requires high current during flight, while avionics and processing units require stable voltage levels. Therefore, the power architecture is designed to support both high-current motor operation and reliable operation of onboard electronic systems.

2.3.10. Architectural Rationale

The ANKEBUT UAV architecture was designed to improve safety, reliability, and maintainability by separating flight-critical control tasks from high-level perception and risk assessment tasks. The Cube Orange flight controller handles stabilization, navigation, motor control, and fail-safe actions, while the Jetson Orin Nano handles image processing, artificial intelligence inference, wind data integration, risk estimation, and alert generation. This separation ensures that the UAV can maintain stable autonomous flight while performing real-time proactive fire risk assessment.

Figures 2.1, 2.2, and 2.3 support this architecture from different perspectives. Figure 2.1 shows the mechanical layout, Figure 2.2 shows the hardware connection structure, and Figure 2.5 shows the software perception and risk assessment pipeline.

2.4. UAV Airframe and Propulsion System

The ANKEBUT UAV is constructed on an S550 hexacopter carbon fiber frame with an approximate motor-to-motor diameter of 550 mm. The hexacopter configuration was selected instead of a quadrotor configuration due to its improved hovering stability, better payload distribution, and greater tolerance to propulsion-related failures. Six SunnySky X4108S 380 KV brushless motors, each paired with 10×4.5 inch carbon fiber propellers,

provide the required thrust for a maximum take-off weight (MTOW) of approximately 2.8–3.0 kg. Motor control is managed by Hobbywing XRotor 40A electronic speed controllers (ESCs). The propulsion system is powered by a 6S 22.2 V, 6,200 mAh, 45C lithium polymer battery, providing an estimated hover endurance of approximately 15–20 minutes under normal operating conditions.

For the propulsion analysis, the upper estimated MTOW value of 3.0 kg was used as a conservative design reference. The total weight force of the UAV can be calculated as:

$$W = m \times g$$

$$W = 3.0 \times 9.81 = 29.43 \text{ N}$$

where W is the total weight force, m is the UAV mass, and g is gravitational acceleration. Since the UAV uses six motors, the minimum theoretical hover thrust required from each motor is:

$$T_{motor} = W / 6$$

$$T_{motor} = 29.43 / 6 = 4.91 \text{ N}$$

This corresponds to approximately 0.50 kgf of thrust per motor under ideal hover conditions. In practice, a higher thrust margin is required to allow stable control, maneuvering, wind resistance, and safe recovery during disturbances. Therefore, the hexacopter configuration provides a more suitable propulsion layout than a quadrotor for carrying the onboard camera, flight controller, communication hardware, and Jetson Orin Nano companion computer.

The total nominal battery energy can be estimated using the battery voltage and capacity:

$$E = V \times Ah$$

$$E = 22.2 \times 6.2 = 137.64 \text{ Wh}$$

where E is the battery energy, V is the nominal battery voltage, and Ah is the battery capacity in ampere-hours. The actual flight endurance depends on the total current consumption of the propulsion system and onboard electronics. The estimated endurance can be expressed as:

$$t = (\text{Battery Capacity} / \text{Average Current}) \times 60$$

Based on the selected battery capacity and the expected current consumption during hover and low-speed scanning missions, the ANKEBUT UAV is expected to achieve approximately 15–20 minutes of hover endurance under normal operating conditions. This value may vary depending on payload weight, wind conditions, throttle level, mission speed, battery health, and safety reserve limits.

Table 0.1 Main airframe and propulsion specifications of the ANKEBUT UAV.

Component / Parameter	*Specification / Value*
Airframe type	*S550 hexacopter carbon fiber frame, 550 mm*
Motors	*6 × SunnySky X4108S 380 KV brushless motors*
Propellers	*10×4.5 inch CW/CCW propellers*
ESCs	*6 × Hobbywing XRotor 40A ESC*
Battery	*6S 22.2 V, 6,200 mAh, 45C Li-Po*
Estimated MTOW	*2.8-3.0 kg*
Weight force at 3.0 kg	*29.43 N*
Minimum hover thrust per motor	*4.91 N ≈ 0.50 kgf*
Estimated hover endurance	*15-20 minutes*

2.5. Software Architecture Overview

The software architecture of the ANKEBUT UAV system is designed as a modular Python-based pipeline running mainly on the NVIDIA Jetson Orin Nano companion computer. This section focuses specifically on the organization of the onboard software components. The software is responsible for receiving camera frames, executing artificial intelligence models, processing telemetry data, retrieving meteorological wind information, generating fire risk outputs and preparing the required data for transmission to the ground control station.

The onboard software is organized into three main functional layers:

- *Perception Layer:* This layer manages RGB frame acquisition, image preprocessing, DeepLabV3-MobileNetV3-Large semantic segmentation and YOLOv11-based detection. It converts raw visual input into meaningful information such as land cover classes and fire/smoke-related detections.
- *Risk Assessment Layer:* This layer combines perception outputs with UAV telemetry, GPS position and wind direction and speed data obtained from the TSMS/MGM API. It estimates the fire spread risk, determines the probable spread direction and prepares geo-referenced risk information.
- *Communication and Output Layer:* This layer manages the exchange of data between the Jetson Orin Nano, Cube Orange flight controller and ground control

station. MAVLink communication over a USB serial connection is used to receive flight telemetry from the Cube Orange, while the wireless communication link is used to transmit live video, telemetry-related information, risk map outputs and generated alerts to the ground station.

This modular software structure allows each part of the system to be developed, tested, and updated separately. It also improves maintainability by separating perception, risk computation, and communication tasks. Detailed explanations of the semantic segmentation model, YOLOv11 detection models, wind-based risk computation and ground control station software are provided in the following sections.

2.5.1. Development and Execution Environment

During development and testing, the software running on the Jetson Orin Nano is accessed and controlled remotely from an Ubuntu laptop using Secure Shell (SSH). In this workflow, the Python scripts and model files remain stored on the Jetson Orin Nano, while the Ubuntu laptop is used to connect to the Jetson, start the required programs, monitor terminal outputs, and debug errors during system testing. This approach allows the team to operate the onboard software without requiring a monitor, keyboard, or mouse to be connected directly to the Jetson during UAV integration.

To establish the SSH connection, the Ubuntu laptop and the Jetson Orin Nano must be connected to the same network and Jetson's local IP address must be identified before running the SSH command. This allows the operator to access the Jetson terminal remotely and execute the required perception, telemetry and risk assessment scripts directly on the onboard mission computer.

The SSH-based workflow is especially useful during field preparation and integration tests because the Jetson Orin Nano can remain mounted on the UAV while the operator runs or stops the perception, telemetry and risk assessment scripts remotely. This setup improves testing efficiency and reflects the practical deployment condition in which the companion computer operates as an onboard mission computer while being supervised from an external development laptop.

2.6. Land Cover Segmentation: Model Design and Training

Land cover segmentation is one of the core components of the ANKEBUT UAV system because the proposed fire risk assessment approach depends on understanding the type of terrain observed from the aerial RGB camera. Instead of relying only on active fire or smoke detection, the system analyzes the scanned area by classifying each image region into meaningful land cover categories such as water, bare soil, green grass, dry grass and trees. These classes provide important information about the relative combustibility of the terrain and allow the system to estimate which areas may contribute more strongly to potential fire spread.

For this purpose, a DeepLabV3 semantic segmentation model with a MobileNetV3-Large encoder backbone was selected. DeepLabV3 was chosen because it is effective for pixel-level scene understanding and can capture contextual information across different spatial scales, which is important for aerial images where vegetation, soil, water and grass regions may appear at different sizes and shapes. The MobileNetV3-Large backbone was selected because it provides a suitable balance between segmentation accuracy and computational efficiency, making it appropriate for real-time inference on the NVIDIA Jetson Orin Nano companion computer.

This section explains the design and training process of the land cover segmentation model, including the model architecture, dataset composition, annotation process, mask preprocessing, training configuration and the mapping between segmentation classes and fire-risk-related combustibility levels.

2.6.1. Architecture: DeepLabV3 with MobileNetV3-Large Encoder

The semantic segmentation model used in the ANKEBUT UAV system is based on the DeepLabV3 architecture **[2]**, with a MobileNetV3-Large encoder backbone **[4]**. The model was implemented using PyTorch's torchvision library through the deeplabv3_mobilenet_v3_large function. Since the implemented model was imported from PyTorch's torchvision library as deeplabv3_mobilenet_v3_large and does not include the additional decoder module associated with DeepLabV3+, it is referred to throughout this thesis as DeepLabV3-MobileNetV3-Large rather than DeepLabV3+.

The model was initialized using torchvision's default pretrained weights, *DeepLabV3_MobileNet_V3_Large_Weights.DEFAULT*, to benefit from feature extraction capabilities learned from large-scale image data. Since the original pretrained

model was not designed for the ANKEBUT UAV land cover classes, the original classification head was replaced with a custom *DeepLabHead(960, 6)*. This modification adapted the network output to the six target classes used in the ANKEBUT UAV dataset:

- Background.
- Water.
- Bare soil.
- Green grass.
- Dry grass.
- Trees.

DeepLabV3 was selected because it is effective for pixel-level scene understanding and can capture contextual information across different spatial scales **[2]**. This capability is important for UAV-based aerial imagery because land cover regions may appear in different sizes, shapes and distributions depending on flight altitude, camera angle and terrain structure. The Atrous Spatial Pyramid Pooling (ASPP) module used in DeepLabV3 applies dilated convolution operations to aggregate multi-scale contextual information, allowing the model to classify both small and large terrain regions more reliably.

The MobileNetV3-Large backbone was selected because of its favorable accuracy-latency trade-off **[4]**. Compared with heavier backbones such as ResNet-50 or ResNet-101, MobileNetV3-Large requires lower computational resources while still providing competitive segmentation performance. This makes it particularly suitable for real-time embedded inference on the NVIDIA Jetson Orin Nano companion computer, where the system must process camera frames during flight while also handling telemetry communication, wind data retrieval and fire risk alert generation.

Overall, *DeepLabV3-MobileNetV3-Large* architecture was selected because it provides a practical balance between segmentation accuracy, computational efficiency and onboard deployment feasibility. Its ability to generate pixel-level land cover predictions from RGB aerial images makes it a core component of the ANKEBUT UAV system's proactive fire risk assessment pipeline.

2.6.2. Dataset Composition, Annotation and Augmentation

The dataset preparation process for the land cover segmentation model consisted of four main stages: dataset preparation and cropping, manual annotation, data augmentation, and

mask preprocessing with dataset splitting. These stages were necessary to convert large aerial and satellite-style images into a structured pixel-level segmentation dataset suitable for training the DeepLabV3-MobileNetV3-Large model.

2.6.2.1. Dataset Preparation and Cropping

The initial dataset was prepared from approximately 200 large RGB satellite-style images collected from publicly available Kaggle image sources. These images contained terrain regions relevant to wildfire risk assessment, including vegetation, grassland, bare soil, and water areas. Since the original images were large and did not directly match the required input size of the segmentation model, a cropping process was applied before annotation and training.

The images were divided into non-overlapping 512×512 pixel patches using an OpenCV-based cropping script. During this process, edge patches smaller than 512×512 pixels were discarded to keep all samples at a fixed resolution. This fixed-size patch format was selected because it is suitable for DeepLabV3 training and makes the images closer to UAV-style aerial views by focusing on smaller terrain regions instead of very large satellite scenes.

After cropping, unsuitable or unclear patches were removed manually. The remaining images were used as the base dataset for annotation and augmentation. This preparation step ensured that the model was trained on consistent image sizes and visually meaningful land cover regions.

2.6.2.2. Manual Annotation Using CVAT

After dataset preparation, the selected 512×512 image patches were manually annotated using the Computer Vision Annotation Tool (CVAT). Since the task is semantic segmentation, annotation was performed using CVAT's polygon-based shape annotation tool rather than bounding boxes. This was necessary because semantic segmentation requires pixel-level masks, where each pixel in the image is assigned to a land cover class. Bounding boxes would only identify the approximate location of an object or region, while polygon annotation allows the exact boundaries of land cover regions to be labeled more accurately.

Each image was labeled at the pixel level according to the six semantic classes used in this thesis: background, water, bare soil, green grass, dry grass, and trees. These classes

were selected because they represent the main land cover categories required for fire risk estimation in the ANKEBUT system. During annotation, polygon shapes were drawn around visible land cover regions, and each polygon was assigned the correct class label, such as dry grass, green grass, bare soil, water, or trees.

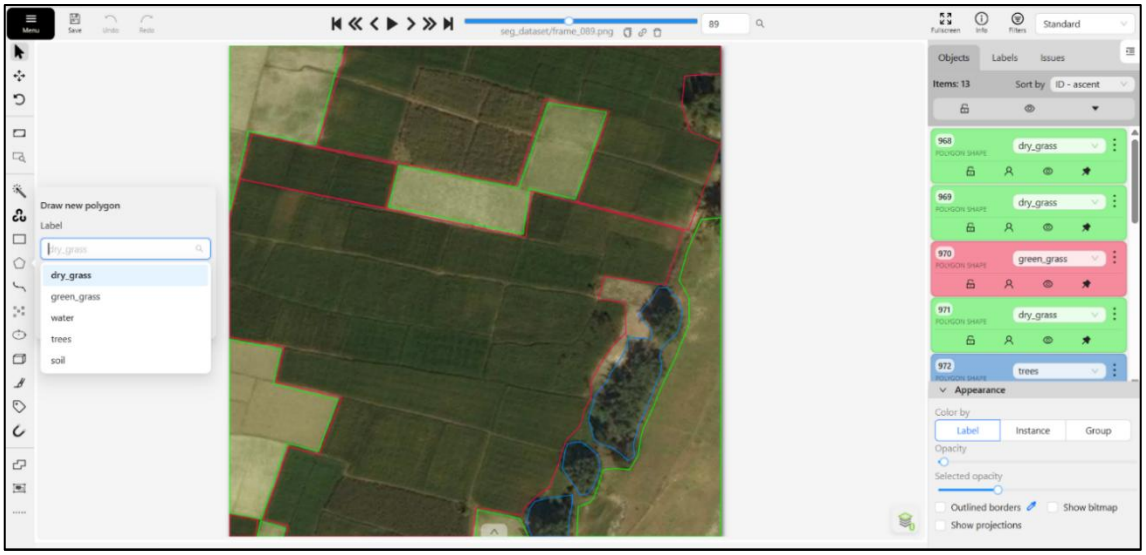


Figure 2. 5 Manual land cover annotation process in CVAT using polygon-based labeling for semantic segmentation mask generation.

The annotation process produced segmentation masks in which each pixel belongs to one of the defined land cover classes. CVAT exported the masks as grayscale PNG files, where each class was represented by a specific grayscale intensity value. In the exported masks, the class pixel values were represented as [0, 109, 113, 169, 199, 213]. These grayscale values were later converted into sequential class indices required by the PyTorch training pipeline.

Figure 2.6 shows an example of the manual annotation process in CVAT using the polygon annotation tool. This figure demonstrates how land cover regions were manually outlined and labeled before being exported as ground-truth masks for model training.

2.6.2.3. Data Augmentation

Data augmentation was applied to increase the number of training samples and improve the visual diversity of the dataset. Since the available manually annotated dataset was limited, augmentation was used to generate additional image-mask pairs from the original annotated samples. The purpose of this step was not to create completely different images, but to produce realistic variations of the same scene so that the model could become more robust to changes that may occur during UAV image acquisition.

The augmentation process was implemented using the Albumentations library. Because this is a semantic segmentation task, augmentation was always applied jointly to each RGB image and its assigned ground-truth mask. In other words, whenever an image was rotated, blurred, or visually modified, the same spatial transformation was applied to its corresponding mask as well. This preserved the pixel-level correspondence between the input image and the annotation mask. Maintaining this alignment is essential, because any mismatch between an image and its assigned mask would negatively affect segmentation training.

The applied augmentations included brightness and contrast adjustment, small-angle rotation, Gaussian blur, motion blur, and slight hue, saturation, and value changes. These transformations were selected because they simulate common visual variations such as changing sunlight conditions, minor viewpoint differences, motion blur caused by UAV movement, and camera-related color variation.

An example of an original aerial image and its augmented version is shown in Figures 2.7 and 2.8. Figure 2.7 presents the augmented image after applying visual transformation operations, while Figure 2.8 presents the original image before augmentation. As shown in the figures, the augmented image remains visually similar to the original image, but small changes in appearance are introduced.



Figure 2.7 Augmented aerial image after applying visual transformation operations.



Figure 2.8 Original aerial image before data augmentation.

These controlled changes help expand the dataset and improve the model's ability to generalize to different aerial imaging conditions while preserving the semantic structure of the original scene.

2.6.2.4. Mask Preprocessing and Dataset Split

To ensure compatibility with PyTorch's loss functions, the grayscale mask values exported from CVAT were converted into contiguous integer class indices starting from zero. Therefore, the original CVAT grayscale pixel value representation {0, 109, 113, 169, 199, 213} was converted into the sequential class index representation {0, 1, 2, 3, 4, 5}. This conversion was necessary because the cross-entropy loss function used during training expects class labels to be represented as integer indices rather than arbitrary grayscale intensity values.

The converted masks were saved in separate output directories to preserve the original CVAT annotation files. After conversion, the dataset was checked by inspecting the unique pixel values present in the mask files. This verification step ensured that all masks contained only valid class indices and that no unexpected pixel values were introduced during preprocessing.

An example of an original aerial image and its corresponding ground-truth segmentation mask is shown in Figures 2.9 and Figure 2.10 show. Figure 2.9 presents the RGB input image used by the model, while Figure 2.10 presents the manually annotated mask representing the land cover classes. The mask visualization uses different colors to distinguish semantic regions such as grass, soil, water, and trees, while the training pipeline internally uses the converted class index values.

After cropping, annotation, augmentation, filtering, and mask preprocessing, the final dataset contained 1,170 image-mask pairs. The dataset was divided into 936 training samples and 234 validation samples, corresponding to an approximate 80%/20% train-validation split. All images and masks were used at a fixed resolution of 512×512 pixels during both training and inference.

No separate held-out test set was defined in this stage. Instead, model selection was performed based on validation loss, and the checkpoint with the lowest validation loss was retained as the best model. Overall, the dataset preparation workflow produced a structured segmentation dataset suitable for training the DeepLabV3-MobileNetV3-Large model. The combination of fixed-size cropping, manual CVAT annotation, controlled augmentation, and mask index conversion allowed the model to learn land cover patterns under different visual conditions while maintaining accurate pixel-level supervision.



Figure 2.11 Original RGB aerial image used as input for the land cover segmentation model.

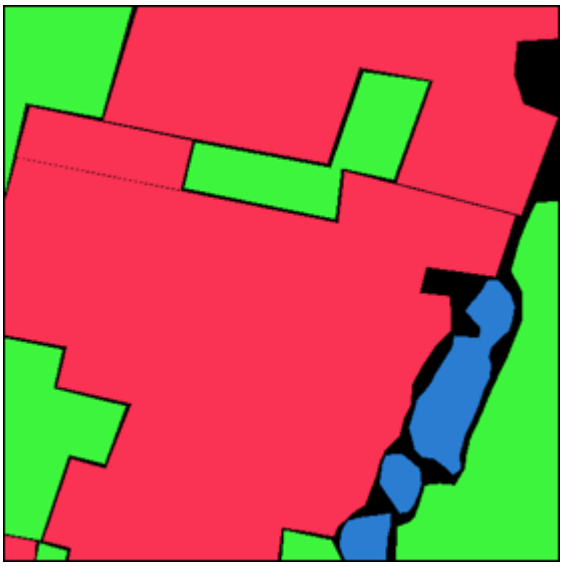


Figure 2.10 Ground-truth semantic segmentation mask corresponding to the original aerial image.

2.6.3. Training Configuration

The DeepLabV3-MobileNetV3-Large segmentation model was trained using the PyTorch framework and torchvision implementation of the model. The training pipeline was designed to process paired RGB images and their corresponding six-class segmentation masks. All images and masks were resized to 512×512 pixels before being passed to the model. RGB images were normalized by scaling pixel values to the range [0, 1], while segmentation masks were loaded as integer class index maps.

Data loading was performed using PyTorch DataLoader objects for both the training and validation splits. The final dataset contained 936 training samples and 234 validation samples. A mini-batch size of 4 was used, and two parallel worker processes were assigned to each DataLoader to improve data loading efficiency during training.

The model was initialized using pretrained torchvision weights and modified for the six-class ANKEBUT land cover segmentation task by replacing the original classification head with a custom DeepLabHead(960, 6). Training was performed for 120 epochs using the Adam optimizer with a constant learning rate of 1×10^{-4}. The cross-entropy loss function was used because the task is a multi-class semantic segmentation problem, where each pixel must be assigned to one of the defined land cover classes. No class-frequency weighting or learning rate scheduling was applied during this training stage.

During each epoch, the model was first trained on the training set and then evaluated on the validation set. Training loss and validation loss were recorded at each epoch to monitor learning progress and identify the best-performing checkpoint. The latest model checkpoint was saved after every epoch, while the best model checkpoint was saved whenever the validation loss improved. The checkpoint achieving the lowest validation loss was retained as the final selected segmentation model.

Table 0.2 Training configuration and validation summary of the DeepLabV3-MobileNetV3-Large segmentation model.

Configuration Item	***Setting / Value***
Framework	*PyTorch / torchvision*
Model architecture	*DeepLabV3-MobileNetV3-Large*
Pretrained weights	*DeepLabV3_MobileNet_V3_Large_Weights.DEFAULT*
Number of output classes	*6*
Input image size	*512×512 pixels*
Training samples	*936*
Validation samples	*234*
Train-validation split	*80% / 20%*
Batch size	*4*
DataLoader workers	*2*
Optimizer	*Adam*
Learning rate	1×10^{-4}
Loss function	*Cross-entropy loss*
Learning rate scheduler	*Not used*
Class weighting	*Not used*
Number of epochs	*120*
Best checkpoint criterion	*Lowest validation loss*
Best validation loss	*0.0856 at epoch 97*
Validation pixel accuracy	*96.85%*

2.6.4. Segmentation Classes and Combustibility Mapping

The six semantic classes used in the ANKEBUT segmentation model correspond to the primary land cover types relevant to fire risk characterization in Turkish forest and

scrubland environments. These classes were selected not only for visual separability in aerial RGB imagery, but also for their relevance to fire spread potential. As summarized in Table 2.3, each land cover class was assigned a combustibility coefficient representing its relative contribution to potential fire spread.

The combustibility coefficients were assigned based on general fire behavior principles and the relative flammability of each land cover type, supported by standard fire behavior literature **[7][1]**. In the ANKEBUT risk assessment pipeline, these coefficients are used as multiplicative weights in the downstream risk fusion computation, where segmentation outputs are combined with wind direction, wind speed, and UAV location data to generate geo-referenced fire risk warnings.

Table 0.3 Semantic segmentation classes and combustibility coefficients used in the ANKEBUT risk assessment pipeline.

Class ID	*Class Name*	*Description*	*Combustibility Coefficient*
0	Background	Unlabeled or irrelevant image regions that are not included in the main land cover categories	*0%*
1	Dry grass	Dry or cured grassland areas with high ignition and rapid fire spread potential	*85%*
2	Green grass	Moist or actively growing grassland areas with lower fire spread potential than dry vegetation	*30%*
3	Water	Rivers, lakes, ponds, irrigation canals, or other water-covered regions that do not burn	*0%*
4	Trees	Tree-covered regions, forest canopy, or dense woody vegetation with high combustibility	*90%*
5	Soil	Bare soil, exposed ground, dirt paths, rocky surfaces, or non-vegetated terrain with very low combustibility	*5%*

2.7. Wind Data Integration and Risk Warning System

The wind data integration and risk warning system is responsible for combining land cover segmentation outputs with meteorological wind information in order to estimate the probable direction and severity of fire spread risk. This section explains how wind speed and wind direction data are retrieved, processed, and fused with segmentation-based combustibility information to generate geo-referenced fire risk warnings.

2.7.1. TSMS / MGM Wind Data Retrieval

Wind direction and wind speed data are retrieved from the Turkish State Meteorological Service, also known as Meteoroloji Genel Müdürlüğü (TSMS/MGM) **[9]**. The purpose of

using TSMS/MGM data is to provide regionally representative meteorological information without requiring an onboard wind sensor. This reduces system cost, weight, and power consumption while still allowing wind-aware fire spread estimation.

During operation, the ground control station retrieves wind information through a Python-based wind data client, using the geographic coordinates of the flagged fire risk location that are transmitted from the UAV. Retrieving the wind data on the ground control station rather than on the onboard companion computer is preferred because the ground station has a reliable internet connection during field operation, whereas the UAV onboard link is dedicated to the drone-to-ground data connection. The wind data include wind direction, represented in degrees from north, and wind speed, represented in meters per second. These values are then passed to the risk assessment module, where they are combined with segmentation outputs and UAV position data. The wind direction is used to estimate the probable direction of fire spread, while wind speed is used to adjust the severity of the risk warning.

Because wind conditions can change over time, the ground control station retrieves updated wind data each time a new fire risk location is reported by the UAV. This allows the risk information to use recent meteorological data for the specific flagged location while avoiding unnecessary repeated API requests. The retrieved wind information is stored and used together with the current UAV location, segmentation map, and detected fire or proxy target position when generating risk warnings.

2.7.2. Risk Fusion Computation

The risk fusion computation combines three main sources of information: the land cover segmentation output, the UAV location data, and the wind direction and speed data retrieved from TSMS/MGM. The segmentation output identifies the type of land cover in each image region, such as dry grass, green grass, trees, soil, water, or background. Each class is assigned a combustibility coefficient, as previously summarized in Table 2.3. These coefficients represent the relative contribution of each class to potential fire spread.

For each segmented region, a base risk score is first assigned according to its combustibility coefficient. Non-combustible or low-risk regions such as water, background, and soil receive low values, while dry grass and trees receive higher values because they are more likely to support ignition and fire spread. The wind vector is then

used to adjust the risk estimate directionally. Regions located in the downwind direction from a detected or suspected ignition source are considered more critical because wind can increase the probability and speed of fire spread in that direction.

The risk score is therefore influenced by both the land cover type and the wind condition. Higher wind speed increases the severity of the warning, while the wind direction determines which areas are more likely to be affected. When a high-risk area exceeds the defined risk threshold, the system generates a geo-referenced warning that includes the estimated location, risk level, and probable spread direction.

The generated warning is transmitted to the ground control station through the communication link and displayed to the operator for mission awareness. The warning may be classified into severity levels such as Moderate, High, or Critical depending on the combustibility of the affected area, the wind speed, and the size of the risky region. This approach allows the ANKEBUT UAV system to move beyond simple fire or smoke detection by producing wind-aware fire spread risk information during autonomous aerial monitoring.

2.8. Fire and Smoke Detection: YOLOv11 Models:

In addition to land cover segmentation, the ANKEBUT UAV system includes an object detection module for identifying fire- and smoke-related visual targets during aerial monitoring. While the DeepLabV3-MobileNetV3-Large segmentation model provides information about terrain type and combustibility, the object detection model provides target-level information by locating visible fire or smoke indicators in the camera stream. These detections support the risk warning pipeline by providing class labels, confidence scores, and bounding box locations that can be combined with UAV telemetry, wind data, and segmentation-based risk information.

YOLOv11-based object detection models were used in this thesis because YOLO architectures are suitable for real-time detection tasks and can process visual input efficiently on embedded GPU platforms such as the NVIDIA Jetson Orin Nano **[5]**. In the ANKEBUT system, two YOLOv11 models were considered. The first model is a production-oriented fire and smoke detection model trained using real fire and smoke images. The second model is a proxy detection model developed for safe system validation, where a red pillow represents fire and a green pillow represents smoke. This two-model approach allows the project to address both the real deployment objective and

the practical limitation of not being able to perform unsafe fire and smoke experiments during controlled field testing.

2.8.1. Purpose of the Detection Module

The purpose of the YOLOv11 detection module is to provide object-level information about visible fire- and smoke-related regions in the UAV camera stream. Unlike semantic segmentation, which assigns a class label to every pixel in the image, object detection identifies target regions using bounding boxes. Each YOLOv11 detection output includes the predicted class, the confidence score, and the bounding box coordinates of the detected object.

In the ANKEBUT UAV system, this information is used to support the fire risk warning pipeline. The detected class indicates whether the observed target is related to fire or smoke. The confidence score represents the model's certainty about the detection, while the bounding box provides the approximate image region where the target appears. When combined with UAV position data and camera-based processing, this information can support the generation of geo-referenced risk alerts.

YOLOv11 was selected because it provides a practical balance between detection accuracy and inference speed. This is important for UAV-based monitoring because the onboard system must process camera frames during flight while also handling telemetry communication, wind data retrieval, segmentation inference, and alert generation. The YOLO-style single-stage detection approach is suitable for real-time applications because it predicts bounding boxes and class probabilities directly in one forward pass.

The detection module is not used as a standalone decision-making component. Instead, it operates together with the land cover segmentation model and the wind-aware risk assessment pipeline. For example, a detected fire or smoke source located near highly combustible land cover classes such as dry grass or trees may produce a higher risk warning than a detection located near water or soil. Therefore, the YOLOv11 detection module strengthens the ANKEBUT system by adding object-level evidence to the segmentation-based fire risk estimation process.

2.8.2. Production Fire and Smoke Detection Model

The production fire and smoke detection model was developed as the real-world detection component of the ANKEBUT UAV system. Its purpose is to detect actual fire and smoke

regions in aerial images and provide object-level evidence for the risk warning pipeline. The model was trained using a YOLOv11 object detection architecture with two target classes:

- Fire
- Smoke

The production dataset consisted of aerial fire and smoke images collected from publicly available sources and organized for object detection training. The images were annotated using bounding boxes, where visible flame regions were labeled as fire and visible smoke regions were labeled as smoke. Fire regions were generally labeled with tighter bounding boxes around the visible flame area, while smoke regions were labeled around visible smoke clouds or plumes.

During the development process, it was observed that smoke detection is more challenging than fire detection because smoke has unclear boundaries, variable transparency, and wide spatial distribution. Unlike fire, which usually appears as a more visually distinct region, smoke may blend with clouds, haze, shadows, or bright background areas. Therefore, the consistency of smoke annotation has a strong effect on the performance of the detection model.

An initial YOLOv11 production model was trained using the prepared fire and smoke dataset. However, the model is still considered a preliminary production model because the smoke class requires further annotation refinement. In particular, smoke regions should be labeled more consistently by using larger bounding boxes for continuous smoke plumes instead of many small boxes inside the same smoke cloud. For this reason, the production fire and smoke model is treated as an ongoing development component, while the safe proxy detection model described in the following section was used for controlled end-to-end system validation.

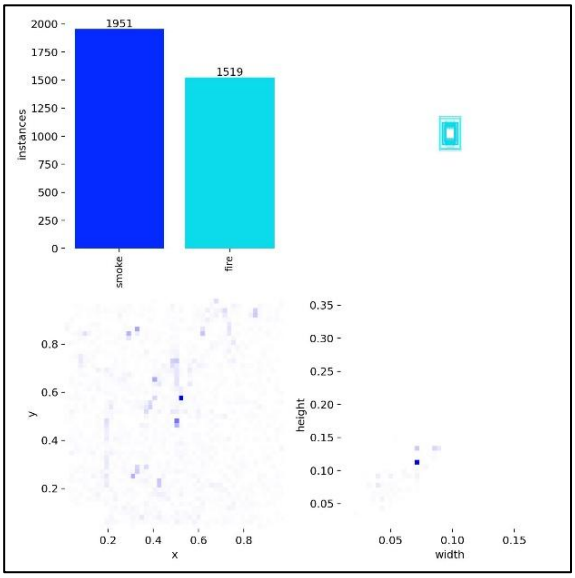


Figure 2. 12 Revised production fire/smoke dataset label distribution after reducing smoke-heavy samples

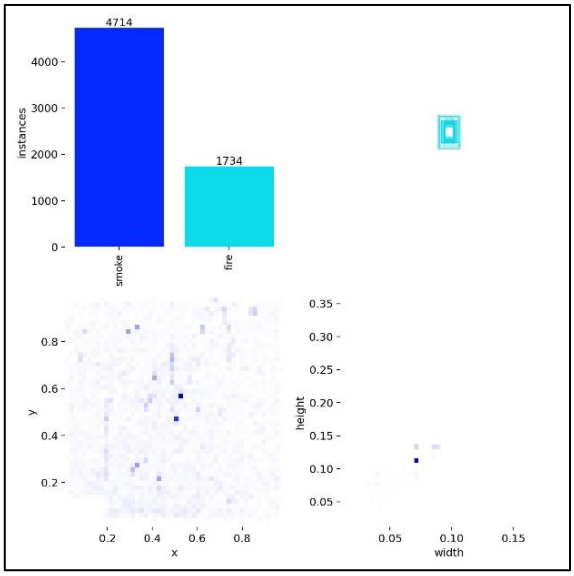


Figure 2. 13 Initial production fire/smoke dataset label distribution before balancing.

The production model remains important for the final deployment objective of the ANKEBUT UAV system. Future improvements will focus on cleaning and standardizing smoke annotations, increasing the number of clear fire examples, improving class balance at the object-instance level, and evaluating whether segmentation-based smoke detection could provide better results than bounding-box detection for diffuse smoke regions.

2.8.3. Dataset Balance and Annotation Refinement for the Production Model

During the development of the production fire and smoke YOLOv11 model, the dataset was analyzed not only according to the number of images, but also according to the number of labeled object instances. This distinction is important in object detection because YOLO learns from bounding box instances rather than image count alone. Therefore, a dataset may appear balanced in terms of image numbers while still being imbalanced at the object-label level.

In the initial version of the production dataset, the smoke class contained a much higher number of labeled instances than the fire class. As shown in Figure 2.12, the initial dataset contained 4714 smoke instances and 1734 fire instances. This means that the smoke class had approximately 2.7 times more labeled objects than the fire class. Such imbalance can affect training behavior and may cause the model to learn one class differently from the other, especially when the visual characteristics of smoke are already more difficult to define than fire.

To reduce this imbalance, smoke-heavy images were reviewed and a portion of the smoke samples was removed from the dataset. The objective was not to make both classes perfectly equal, but to reduce the large difference between smoke and fire object instances while preserving useful smoke diversity. After this correction, the revised dataset contained 1951 smoke instances and 1519 fire instances, as shown in Figure 2.11. This reduced the class ratio to approximately 1.3:1, which is more suitable for training than the initial distribution.

Table 0.4 Production fire/smoke dataset balance before and after reducing smoke-heavy samples.

Dataset Version	***Smoke Instances***	***Fire Instances***	***Approximate Ratio***	***Description***
Initial dataset	4714	1734	2.7:1	*Smoke-heavy dataset with strong object-instance imbalance*
Revised dataset	1951	1519	1.3:1	*Reduced smoke-heavy samples for improved class balance*

Although the revised dataset improved the object-instance balance, smoke detection remained more challenging than fire detection due to the nature of smoke itself. Smoke regions are often semi-transparent, irregular, and spread over wide areas. In addition, smoke boundaries are less clearly defined than flame boundaries, making bounding box annotation less straightforward. For this reason, future improvements to the production model will focus not only on class balance, but also on improving annotation consistency for smoke regions.

In particular, continuous smoke plumes should be labeled using consistent bounding boxes that cover the main visible smoke region, instead of using many small boxes inside the same cloud. This refinement is expected to improve the training quality of the production fire and smoke detector. Therefore, the production model is considered an ongoing development component, while the proxy pillow detection model is used for safe and controlled end-to-end validation in this thesis.

2.8.4. Proxy Pillow Detection Model for Safe Validation

Since real fire and smoke testing could not be performed safely in the workshop or field environment, a proxy dataset was prepared to validate the detection pipeline under controlled conditions. In this dataset, two physical objects were used to represent fire- and smoke-related targets. A red pillow was used as the fire proxy, while a green pillow was used as the smoke proxy. This approach allowed the system to be tested safely without creating real fire, smoke, or environmental risk.

The proxy dataset was collected in the workshop environment using both the UAV camera and a mobile phone camera. Images were captured from different distances, heights, and viewing angles to simulate the visual variation that may occur during UAV-based monitoring. The pillows were placed in different positions within the scene so that the model could learn to detect them under different spatial conditions and backgrounds.

The collected images were annotated using bounding boxes because YOLOv11 is an object detection model. Unlike the segmentation dataset, where polygon annotations were used to create pixel-level masks, the YOLO dataset required rectangular bounding boxes around each target object. During annotation, each red pillow was labeled as the fire proxy class, and each green pillow was labeled as the smoke proxy class. Figure 2.13 shows an example from the proxy pillow dataset and demonstrates the bounding box annotation process used in CVAT.

After annotation, the dataset was exported in YOLO format, where each image has a corresponding text label file containing the class ID and normalized bounding box coordinates. This format allows the YOLOv11 training pipeline to read the image-label pairs directly during training.

The purpose of this proxy dataset was not to replace the real fire and smoke dataset. Instead, it was used to create a safe and repeatable validation scenario for the complete ANKEBUT detection workflow. By using clear and controlled proxy objects, the team could test whether the Jetson Orin Nano could process camera frames, run YOLOv11 inference, identify the target class, and generate outputs that can be used by the alert generation pipeline.

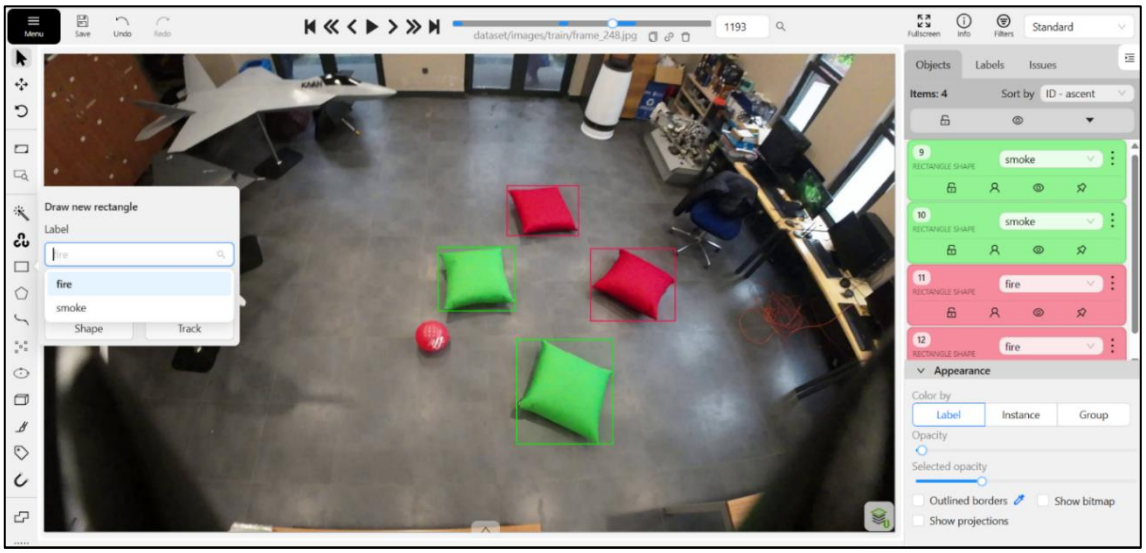


Figure 2. 14 Bounding box annotation example from the proxy pillow dataset in CVAT.

2.8.5. Proxy YOLOv11 Training Configuration

The proxy pillow detection model was trained using the Ultralytics YOLOv11 object detection framework. The model was initialized using the pretrained YOLOv11 nano checkpoint, which provides lightweight architecture suitable for real-time object detection and later deployment on embedded hardware such as the NVIDIA Jetson Orin Nano. The training task was defined as object detection, where each target is represented by a bounding box and a class label.

The proxy dataset was exported in YOLO format after annotation. In this format, each image is paired with a corresponding text label file containing the class ID and normalized bounding box coordinates. The input images were resized to 640×640 pixels during training. The model was trained using a batch size of 16 for a planned training duration

of 200 epochs. During training, YOLO automatically monitored standard object detection metrics, including precision, recall, mAP@0.5, and mAP@0.5:0.95.

The training process used pretrained weights and the default Ultralytics optimization settings. Data augmentation was also applied by the YOLO training pipeline, including common transformations such as scaling, translation, horizontal flipping, mosaic augmentation, and color variation. These transformations helped increase robustness by exposing the model to different object positions, scales, lighting conditions, and viewing angles.

Table 0.5 Training configuration of the proxy YOLOv11 detection model.

Configuration Item	***Setting / Value***
Framework	Ultralytics YOLOv11
Model architecture	YOLOv11 nano object detection model
Initial weights	Pretrained YOLOv11 nano weights
Task type	Object detection
Dataset format	YOLO bounding box format
Number of classes	2 proxy classes
Class names	Fire and Smoke
Input image size	640×640 pixels
Batch size	16
Planned number of epochs	200
Optimizer	Ultralytics automatic optimizer
Evaluation metrics	Precision, recall, mAP@0.5, mAP@0.5:0.95
Best checkpoint criterion	Best validation performance

In the recorded training run, the proxy model reached strong validation performance. The recorded results showed high precision and recall, indicating that the model was able to detect the proxy targets reliably under the controlled workshop conditions. The final recorded validation results are discussed in the following subsection.

2.8.6. Proxy Model Results and Validation

The trained proxy YOLOv11 model was evaluated on the validation set using standard object detection metrics, including precision, recall, mAP@0.5, and mAP@0.5:0.95. These metrics were used to measure how reliably the model detected the proxy fire and smoke targets under controlled workshop conditions.

The recorded validation results showed that the proxy model achieved strong detection performance. The final validation precision reached 0.997, while the recall reached 0.976.

The model also achieved an mAP@0.5 value of 0.987 and an mAP@0.5:0.95 value of 0.839. These results indicate that the proxy model was able to detect the target objects accurately and consistently in the validation images.

The high validation performance of the proxy model demonstrates that the detection pipeline can operate reliably under safe and repeatable testing conditions. Although the proxy model does not replace the production fire and smoke detection model, it validates the practical operation of the object detection stage. Therefore, the proxy YOLOv11 model was used to confirm that the ANKEBUT system can process camera frames, detect target objects, and provide detection outputs suitable for the downstream alert generation pipeline.

2.8.7. Integration with the ANKEBUT Alert Pipeline

The YOLOv11 detection module is integrated into the ANKEBUT alert pipeline as an object-level evidence source. During operation, the Jetson Orin Nano receives frames from the onboard RGB camera and processes them through the trained YOLOv11 model. The model produces detection outputs containing the predicted class label, confidence score, and bounding box coordinates for each detected target.

These detection outputs are used together with the land cover segmentation result, UAV telemetry, GPS position, and wind data. The detected object class indicates whether a fire- or smoke-related target is present in the image, while the bounding box location identifies the approximate image region where the target appears. This information can then be associated with the UAV position and used by the risk assessment module to generate more informative warning outputs.

In the full ANKEBUT system, the detection output is not treated as an independent final decision. Instead, it supports the risk fusion process. For example, if a fire- or smoke-related detection appears near land cover regions classified as dry grass or trees, the generated warning may be considered more critical than a detection located near water or soil. Similarly, wind direction and wind speed are used to estimate the probable spread direction and severity of the detected risk.

After the detection and risk fusion stages, the system generates an alert containing the detected class, confidence level, estimated risk location, and probable spread direction. This information is transmitted to the ground control station for operator monitoring. In

this way, the YOLOv11 module contributes to the complete detection-to-alert workflow by converting camera-based visual information into actionable warning data.

The proxy YOLOv11 model was used to validate this workflow safely. By detecting the red and green pillow targets under controlled conditions, the system confirmed that the Jetson Orin Nano could run object detection, produce class and location outputs, and support the downstream alert generation process without requiring unsafe real fire or smoke experiments.

2.9. Ground Control Station Software

The ANKEBUT Ground Control Station (GCS) is a custom desktop application developed in Python 3.10 using the PySide6 framework, running on a Windows 11 host machine. The GCS provides the operator interface for mission monitoring, real-time telemetry display, live video reception, and integration with Mission Planner for flight control. The application is organised as a modular package under the ankebut_gcs directory, with clearly separated concerns across the ui/, comms/, and simulator/ submodules.

2.9.1. Architectural Design Philosophy

The GCS architecture follows a hybrid integration principle: rather than reimplementing flight-control functionality already available in Mission Planner, the custom application embeds Mission Planner as a native window panel and extends it with two original components, a live video panel and a telemetry display panel. This approach eliminates duplication of battle-tested MAVLink parsing and mission management logic, allowing development effort to focus exclusively on the novel contributions of real-time video reception and AI result display.

The application window is sized at 1600 × 900 pixels. The layout places Mission Planner in the left two-thirds of the window at full height, with the video panel in the upper-right quadrant and the telemetry panel in the lower-right quadrant.

The choice to embed Mission Planner rather than develop a fully custom GCS from scratch is grounded in engineering pragmatism. Mission Planner is a mature, well-documented, and actively maintained open-source application that implements the complete MAVLink command and telemetry protocol, provides a high-quality moving map with waypoint overlay, and includes parameter tuning, pre-arm checks, and flight

log analysis tools. Replicating this functionality would require a substantial and error-prone development effort without adding value to the project's core research contributions. By treating Mission Planner as a black-box sub-application and embedding it via the Win32 windowing subsystem, the custom GCS inherits all of these capabilities at zero additional cost while remaining free to innovate in the areas that Mission Planner does not cover: live compressed video display and structured AI detection result presentation.

The modular Python package structure (ankebut_gcs/) enforces a clean separation of concerns. User interface components reside in the ui/ subpackage and communicate exclusively through Qt signals and slots. Network communication components are isolated in the comms/ subpackage, where each channel (video UDP stream, MAVLink UDP, future AI JSON channel) is managed by an independent QThread subclass so that packet loss or latency on one channel cannot block reception on another. A simulator/ subpackage provides hardware-free drop-in replacements for each live data source, enabling full GUI development and testing without a physical drone.

2.9.2. Mission Planner Integration

Mission Planner is embedded into the PySide6 application window using the Windows Win32 API. At runtime, the MissionPlannerHost module enumerates all active windows via win32gui.EnumWindows, identifies the Mission Planner window by title string matching, removes its native title bar and resize border by clearing the WS_CAPTION and WS_THICKFRAME window style flags via win32gui.SetWindowLong, and wraps the resulting window handle into a Qt container widget using QWindow.fromWinId() and QWidget.createWindowContainer(). This technique allows Mission Planner to operate fully within its own process, maintaining all ArduPilot MAVLink functionality, map rendering, and waypoint management, while appearing visually integrated into the custom GCS interface.

The embedding procedure is triggered by the operator clicking the Embed Mission Planner button in the GCS interface after Mission Planner has been launched as a separate process. Once triggered, win32gui.EnumWindows iterates over all top-level windows in the desktop session and compares each title string against the expected Mission Planner substring, making the detection robust to version-dependent title variations.

Upon finding the target window, the module records its Win32 window handle (HWND). The WS_CAPTION and WS_THICKFRAME flags are cleared via win32gui.SetWindowLong, giving Mission Planner a borderless appearance appropriate for embedding. The HWND is passed to QWindow.fromWinId(), which creates a Qt-managed wrapper, then converted by QWidget.createWindowContainer() into a standard QWidget placed in the main QSplitter as the left panel.

An important operational consideration is that Mission Planner continues to run as a fully independent Windows process throughout the session. Its event loop, rendering, and serial or UDP communication with the autopilot are entirely unaffected by the embedding. If Mission Planner is closed or crashes, the GCS restores the placeholder panel with a re-embed button, allowing the operator to restart Mission Planner and re-attach without restarting the custom GCS application.

2.9.3. Real-Time Video Reception Pipeline

Live video from the drone is transmitted as an MPEG-TS stream encoded with H.264, delivered over UDP on port 5601. The video reception pipeline is implemented in the VideoPanel and UdpVideoReceiver classes. The UdpVideoReceiver runs as a QThread to prevent blocking the main GUI event loop. Video frames are decoded using OpenCV's VideoCapture interface with the FFmpeg backend, opening the UDP stream as udp://@0.0.0.0:5601. To minimise end-to-end latency, the following FFmpeg options are applied: fflags=nobuffer, flags=low_delay, probesize=32768, and analyzeduration=0. Decoded frames are converted from BGR to RGB, wrapped as QImage objects, and emitted via a Qt signal to the VideoPanel widget for display.

The choice of MPEG-TS as the container format over alternatives such as RTP/H.264 or raw UDP is deliberate. MPEG-TS is a streaming-oriented container that provides byte-level synchronisation markers, allowing a receiver to lock onto the stream mid-flow without requiring the beginning of the file. This property is essential in a UAV context where the GCS may be started after the drone has already begun transmitting, or where brief packet loss must not cause permanent desynchronisation. The FFmpeg backend in OpenCV's VideoCapture automatically handles MPEG-TS parsing and H.264 decoding, abstracting the complexity of the transport layer from the application code.

The GStreamer pipeline on the Jetson side is constructed to minimise encoding latency. The x264enc encoder is configured with tune=zerolatency, which disables lookahead

buffering and forces the encoder to produce output frames as soon as input frames are received, at the cost of a slight reduction in compression efficiency. The key-int-max=15 parameter ensures that an IDR (Intra-coded) frame is inserted at least every 15 frames (0.5 seconds at 30 fps), reducing the time required for a receiver to achieve full decoding synchronisation after joining the stream or recovering from packet loss. Buffer sizes on both the sender and receiver are minimised: the udpsink buffer-size is set to 65535 bytes (one UDP datagram) and the leaky=downstream queue elements on the sender side drop frames rather than accumulate backlog, ensuring that the GCS always receives the most recent frame rather than a queue of stale frames.

On the receiver side, the UdpVideoReceiver QThread calls cv2.VideoCapture.read() in a tight loop. When a frame is successfully decoded, it is immediately converted from OpenCV's native BGR colour order to Qt's expected RGB order using cv2.cvtColor, wrapped in a QImage with the QImage.Format_RGB888 format, and emitted via the frame_received Qt signal. The VideoPanel widget's slot receives this signal on the main GUI thread and scales the QImage to fit the panel dimensions using Qt.KeepAspectRatio scaling before drawing it to the QLabel. This design ensures that heavy decoding work occurs off the main thread while the display update occurs safely on the main thread, in accordance with Qt's threading model.

Table 0.6 GCS video reception pipeline configuration parameters.

Parameter	*Value*
Transport protocol	*UDP unicast*
Stream format	*MPEG-TS over UDP*
Video codec	*H.264 (x264enc / openh264enc)*
Receive port	*5601*
Resolution	*1280 × 720 pixels*
Target frame rate	*30 fps*
Decoder backend	*OpenCV VideoCapture with FFmpeg (cv2.CAP_FFMPEG)*
Latency flags	*fflags=nobuffer, flags=low_delay, probesize=32768, analyzeduration=0*

2.9.4. Wireless Communication Infrastructure

The wireless link between the Jetson Orin Nano and the ground station is established using a Realtek RTL8812AU dual-band USB WiFi adapter (Alfa AWUS036ACH) operating in Access Point (AP) mode on the Jetson. The RTL8812AU driver (88XXau, aircrack-ng) was compiled and installed on JetPack 6.2.1. The Jetson is configured as a 5 GHz 802.11ac access point using hostapd, broadcasting under the SSID

ANKEBUT_VIDEO on channel 36 (5180 MHz) with WPA2 authentication. IP address assignment is managed by dnsmasq providing DHCP service in the 192.168.10.0/24 subnet. The Jetson holds the static IP 192.168.10.1 and the ground station connects as a client at 192.168.10.20. This dedicated link isolates video and telemetry traffic from external networks, reducing latency variability and packet loss.

The selection of 5 GHz over 2.4 GHz for the video link is motivated by two factors. First, the 5 GHz band (specifically the UNII-1 channels 36–48) is significantly less congested than 2.4 GHz in typical field environments, where competition from consumer WiFi, Bluetooth, and ZigBee devices can cause substantial packet loss at 2.4 GHz. Second, the 802.11ac standard available on 5 GHz supports higher physical-layer data rates than 802.11n, providing adequate headroom for H.264-encoded video at 2–4 Mbps with substantial margin. Channel 36 (5180 MHz) was selected as the lowest available UNII-1 channel, minimising interference from the upper UNII-2 dynamic frequency selection (DFS) channels that require radar detection compliance.

The WPA2 security configuration uses a pre-shared key (PSK) with AES-CCMP encryption, providing adequate protection against passive eavesdropping in the field environment. The DHCP service provided by dnsmasq assigns addresses in the 192.168.10.100–192.168.10.200 range to connecting clients, while the Jetson's own interface is assigned the static address 192.168.10.1. The ground station PC's Alfa adapter is configured with a static IP of 192.168.10.20 outside the DHCP range, preventing address conflicts. The dedicated subnet (192.168.10.0/24) ensures that video and telemetry traffic is fully isolated from any other network infrastructure present in the operational environment.

A complementary 433 MHz SiK radio link provides the telemetry channel between the Cube Orange flight controller and the ground station. This link operates independently of the WiFi video channel and carries MAVLink messages including GPS position, attitude, battery state, and autopilot status at a data rate of 57.6 kbps. The SiK radio's low frequency provides substantially better obstacle penetration and range than the 5 GHz WiFi video link, ensuring that flight control telemetry remains available even when the video link is obstructed. This dual-channel architecture means that loss of video does not equate to loss of situational awareness: the operator retains full telemetry visibility and the ability to issue MAVLink commands via Mission Planner even when the video panel is blank.

Table 0.7 Jetson Orin Nano wireless access point configuration.

Parameter	*Value*
WiFi adapter	Alfa AWUS036ACH (RTL8812AU chipset)
Driver	88XXau (aircrack-ng)
Operating mode	Access Point (hostapd)
SSID	ANKEBUT_VIDEO
Frequency band	5 GHz (802.11ac), channel 36 (5180 MHz)
Security	WPA2
DHCP server	dnsmasq
Jetson static IP	192.168.10.1
Ground station IP	192.168.10.20

2.9.5. MAVLink Telemetry Reception

The GCS receives MAVLink telemetry from the drone via a QThread-based mavlink_receiver module listening on UDP port 14551 using the pymavlink library, parsing message types including HEARTBEAT, SYS_STATUS, GLOBAL_POSITION_INT, ATTITUDE, and VFR_HUD. Parsed values are exposed through a TelemetryProvider interface that mirrors the mock_telemetry module used during offline development, ensuring that switching between simulated and live telemetry requires only a single-line import change. The telemetry panel displays altitude, groundspeed, heading, battery voltage, and GPS fix status, updated at a 250 ms refresh interval. Mission Planner independently receives MAVLink data on port 14550, providing the primary flight control interface; the custom telemetry panel provides a compact always-visible status summary.

The HEARTBEAT message is parsed to monitor autopilot mode (e.g., AUTO, GUIDED, LOITER, RTH) and arm state, enabling the telemetry panel to display a colour-coded mode indicator. The GLOBAL_POSITION_INT message provides latitude, longitude, and altitude with centimetre-level resolution, which the panel displays as decimal degrees and metres above home. The ATTITUDE message delivers roll, pitch, and yaw angles in radians, converted to degrees for display. The VFR_HUD message provides groundspeed, airspeed, climb rate, and throttle percentage as a compact flight instrument summary.

The TelemetryProvider abstract interface defines a single method signature for each telemetry parameter, allowing the main window and telemetry panel to retrieve values without knowledge of whether the data originates from the live mavlink_receiver, the mock_telemetry simulator, or any future alternative source. This pattern is equivalent to

the Strategy design pattern and ensures that the display layer is never modified when the data source changes. During the development phase, the drone_mavlink_sender.py simulator script in the simulator/ subpackage broadcasts all required MAVLink message types over UDP, enabling full GUI testing on a development workstation with no physical hardware connected.

2.9.6. GCS Software Module Summary

To summarize the software organization of the ANKEBUT Ground Control Station, Table 2.8 lists the main software modules, their corresponding implementation files, and their primary roles in the application. This table provides a compact overview of how the GCS software is divided into user interface, video reception, telemetry communication, configuration, and simulation components.

Table 0.8 GCS software modules and their roles.

Module	File	Function
Main window	ui/main_window.py	Root layout: MP panel (left), video + telemetry (right)
MP host	ui/mission_planner_host.py	Win32 API embedding of Mission Planner window
Video panel	ui/video_widget.py	UDP MPEG-TS H.264 reception and display
Telemetry panel	ui/telemetry_panel.py	Live MAVLink telemetry display widget
MAVLink receiver	comms/mavlink_receiver.py	QThread pymavlink UDP listener (port 14551)
Configuration	config.py	Centralised port, window, and connection constants
Video simulator	simulator/drone_video_sender.py	Webcam-to-MPEG-TS UDP test sender

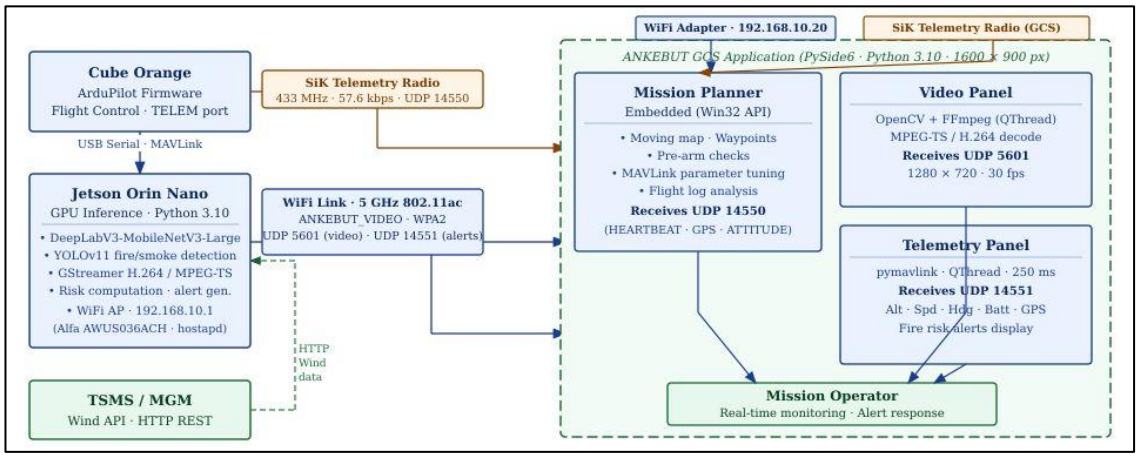


Figure 2.9 Ground control station software and communication architecture: PySide6 application layout with embedded Mission Planner, live video panel, and telemetry panel.

2.9.7. Communication Performance and Link Reliability

The dual-channel communication architecture was evaluated through a structured series of range and reliability tests prior to integration with the full UAV system. In the initial PC-to-PC bench test, the GStreamer video sender and the GCS receiver were operated on two laptop computers connected to a standard 802.11ac access point. Continuous video streaming at 1280×720 pixels and 30 fps was sustained for periods exceeding 15 minutes with zero frame loss, validating the correctness of the MPEG-TS pipeline and the FFmpeg receive configuration.

The Jetson-to-PC over-the-air test was conducted with the Jetson configured as the AP and the ground station PC connected as a client. Video streaming from the Jetson's camera over the ANKEBUT_DRONE SSID at distances up to 30 metres in an open outdoor environment was sustained without visible artefacts. At 50 metres with partial line-of-sight obstruction (a single wall), occasional brief pixelation was observed but frame delivery was maintained. The MPEG-TS transport layer's mid-stream synchronisation capability ensured that recovery from brief packet bursts was automatic and transparent to the operator.

The SiK 433 MHz telemetry link maintained uninterrupted MAVLink connectivity throughout all test scenarios, including the PC-to-PC video stress tests in which the 5 GHz channel was deliberately saturated. This confirmed that the telemetry and video channels are electrically and spectrally independent, and that degradation of one does not affect the other. End-to-end alert latency from proxy detection on the Jetson to notification display on the GCS was measured at under 2 seconds across all test scenarios, meeting the system's defined operational target.

2.9.8. Operator Interface and Usability Considerations

The GCS user interface was designed with operator workload and situational awareness in mind. The 1600×900-pixel window allocates approximately two-thirds of its area to the embedded Mission Planner panel, providing unobstructed access to the moving map, waypoint list, and autopilot status indicators that are central to mission monitoring. The upper-right video panel is sized at approximately 640×480 pixels, sufficient to resolve the bounding box overlays that will be rendered by the AI detection pipeline in future integration phases. The lower-right telemetry panel presents a condensed summary of the six most operationally critical parameters, altitude, groundspeed, heading, battery

voltage, GPS fix quality, and autopilot mode, in a format readable at a glance without requiring the operator to navigate Mission Planner's more detailed but denser telemetry widgets.

Informal feedback collected during integration tests indicated that operators found the hybrid layout intuitive. The embedded Mission Planner interface required no additional training beyond familiarity with standalone Mission Planner. The custom video and telemetry panels were described as a useful supplement that reduced the need to switch between application windows during a mission. Future enhancements identified during testing include adding a dedicated AI alert notification area in the lower-right region, an audio alarm for Critical-level risk events, and a mission log export function accessible from the GCS toolbar.

3. RESULTS AND DISCUSSION

This chapter presents the results obtained from the implementation, training, validation, and integration of the ANKEBUT UAV system. The results are organized according to the main components of the project: UAV hardware integration, autonomous mission execution, land cover segmentation, YOLOv11-based proxy detection, wind-aware risk assessment, ground control station operation, and end-to-end system validation.

The main objective of this chapter is to evaluate whether the developed system can support proactive UAV-based forest fire risk monitoring. Therefore, the discussion focuses not only on individual model performance, but also on the interaction between perception, telemetry, wind data, communication, and operator monitoring. The results demonstrate how the ANKEBUT system combines onboard artificial intelligence, autonomous flight control, meteorological data integration, and ground station supervision within a single operational workflow.

3.1. System Implementation Overview

The ANKEBUT UAV system was implemented as an integrated aerial monitoring platform designed to support proactive forest fire risk assessment. The system combines a Cube Orange flight controller, NVIDIA Jetson Orin Nano companion computer, RGB camera, GNSS/GPS module, telemetry link, wireless video communication, and a custom ground control station interface. Each subsystem contributes to a specific part of the overall workflow, from autonomous flight execution to AI-based perception and operator alert delivery.

During operation, the UAV follows a pre-planned grid-based mission uploaded through the ground control station. The Cube Orange flight controller manages flight stabilization, waypoint navigation, flight modes, and failsafe behavior, while the Jetson Orin Nano performs the onboard mission-computer tasks. These tasks include receiving camera frames, running the DeepLabV3-MobileNetV3-Large segmentation model, running the YOLOv11 proxy detection model during safe validation tests, and preparing risk outputs for communication to the ground station, where the corresponding wind data for the flagged location is retrieved.

The perception pipeline was implemented to provide two types of visual information. The first is land cover segmentation, where the camera image is classified into terrain classes such as dry grass, green grass, water, trees, soil, and background. This result is used to

estimate the relative combustibility of the observed area. The second is object detection, where YOLOv11 is used to detect fire- or smoke-related targets. Since real fire and smoke testing was not safe during system validation, a proxy pillow detection model was used to test the detection-to-alert workflow under controlled conditions.

The ground control station provides the operator with mission monitoring and system feedback. It integrates Mission Planner for autopilot interaction and adds custom panels for live video, telemetry summary, and alert awareness. Communication between the UAV and the ground station is maintained through separate video and telemetry channels, allowing the operator to monitor flight state and visual information during system tests.

Overall, the implemented system demonstrates the feasibility of combining autonomous UAV operation, onboard AI inference, wind-aware risk estimation, and operator supervision. The following sections present the results of the main system components, including mission performance, segmentation performance, YOLOv11 proxy detection results, risk warning behavior, and ground control station integration.

3.2. UAV Hardware Integration and Mission Performance

The ANKEBUT UAV hardware was integrated and tested to verify that the airframe, propulsion system, flight controller, companion computer, camera, telemetry link, and ground control station could operate together as a complete aerial monitoring platform. The S550 hexacopter frame provided sufficient mounting space for the Cube Orange flight controller, Jetson Orin Nano companion computer, RGB camera, GPS module, telemetry equipment, and power distribution components.

During integration, the Cube Orange flight controller was responsible for flight stabilization, waypoint navigation, flight mode management, and failsafe behavior. The Jetson Orin Nano operated as the onboard mission computer and was responsible for perception, AI inference, risk-related processing, and communication with the ground station. Separating the flight-control function from the AI-processing function improved system reliability because flight-critical stabilization remained under the control of the Cube Orange, while computationally intensive tasks were handled by the companion computer.

Autonomous mission behavior was evaluated using pre-planned grid-based waypoint missions uploaded through Mission Planner. The grid mission structure allowed the UAV to scan the selected area systematically while maintaining operator supervision through

telemetry and ground station monitoring. During the tests, the UAV was able to follow the planned waypoint sequence, maintain stable flight, and support Return-to-Launch behavior according to the configured safety parameters.

The results showed that the UAV platform could support autonomous grid-based mission execution while carrying the required onboard electronics. The estimated flight endurance of approximately 15-20 minutes was suitable for short aerial monitoring sorties and field validation tests. Since the Jetson Orin Nano was powered separately from the main propulsion battery using a dedicated 3S 1800 mAh LiPo battery, the companion computer could operate independently from the main motor power path.

Table 0.1 UAV hardware integration and mission performance indicators observed during system testing.

Performance Indicator	Observed Result	Notes
Mission type	Pre-planned grid-based waypoint mission	Uploaded through Mission Planner
Area coverage	More than 90% of planned waypoints reached	Minor incomplete coverage may occur when battery failsafe is triggered
Estimated endurance	15-20 minutes	Based on 6S 6200 mAh LiPo flight battery under representative payload conditions
Flight controller	Cube Orange	Used for stabilization, waypoint navigation, and failsafe behavior
Companion computer	NVIDIA Jetson Orin Nano	Used for onboard AI inference and mission-computer tasks
Telemetry communication	MAVLink telemetry link	Used for flight state monitoring and ground station communication
Return-to-Launch behavior	Verified during mission testing	Triggered according to configured failsafe conditions
Operator supervision	Available through GCS	Mission Planner, telemetry, video, and alert information support monitoring

No major instability was observed during the tested mission conditions. The telemetry link allowed the operator to monitor flight status, while the ground control station supported mission supervision. The results confirm that the selected hardware configuration is suitable for validating the ANKEBUT UAV concept under controlled test conditions.

3.3. Land Cover Segmentation Results

This section presents the training and validation results of the DeepLabV3-MobileNetV3-Large semantic segmentation model used for land cover classification in the ANKEBUT UAV system. The model was trained to classify aerial RGB images into six semantic

classes: background, dry grass, green grass, water, trees, and soil. These segmentation outputs form the basis of the combustibility mapping stage used later in the wind-aware fire risk assessment pipeline.

The model was trained for 120 epochs using 936 training images and evaluated on 234 validation images at the end of each epoch. Training and validation losses were recorded throughout the process in order to monitor model convergence and identify the best-performing checkpoint. Table 3.2 presents selected training and validation loss values from the training process, while Figure 3.2 presents the full training and validation loss curves over 120 epochs.

The training loss generally decreased from 0.7995 at epoch 1 to 0.0521 at epoch 120, showing that the model continued learning throughout the training process. The validation loss decreased rapidly during the early training epochs and then improved more gradually. The lowest validation loss was obtained at epoch 97, where the validation loss reached 0.0856. Although training continued until epoch 120, the best checkpoint was selected at epoch 97 because it achieved the lowest validation loss. After this point, the validation loss showed only minor fluctuations, indicating that the model had largely converged.

Table 0.2 Selected training and validation loss values of the DeepLabV3-MobileNetV3-Large segmentation model.

Epoch	Training Loss	Validation Loss	Checkpoint Status
1	*0.7995*	0.6441	First epoch
5	0.2799	0.2428	Improved
10	0.1949	0.1769	Improved
20	0.1349	0.1322	Improved
35	0.0964	0.1045	Improved
50	0.0757	0.0947	Improved
63	0.0681	0.0906	Improved
80	0.0619	0.0877	Improved
93	0.0569	0.0871	Improved
97	0.0604	0.0856	Best checkpoint
110	0.0542	0.0869	Not selected
120	0.0521	0.0875	Last epoch

Table 0.3 Final validation performance of the DeepLabV3-MobileNetV3-Large segmentation model.

Metric	Value
Best validation loss	*0.0856*
Best checkpoint epoch	97
Final training loss at epoch 120	0.0521

Validation pixel accuracy	96.85%
Training samples	936
Validation samples	234
Total training epochs	120
Training platform	Google Colab, NVIDIA A100 GPU

The selected model achieved an overall validation pixel accuracy of 96.85%. This value represents validation pixel accuracy rather than test accuracy because no separate held-out test set was defined for this stage. The result indicates that the model correctly classified the majority of pixels in the validation images and provides a strong basis for the downstream combustibility and risk assessment pipeline.

A qualitative validation example is shown in Figure 3.1. The figure includes the input RGB image, the corresponding ground-truth segmentation mask, and the predicted segmentation mask generated by the trained model. Visual inspection shows that the model is able to identify the main land cover regions with good spatial consistency. Large regions such as water, soil, grass, and vegetation are generally segmented clearly. Some confusion may still occur between visually similar vegetation classes, especially green grass and dry grass, because their appearance can vary depending on lighting conditions, image quality, and terrain texture.

Overall, the segmentation results demonstrate that the DeepLabV3-MobileNetV3-Large model is suitable for the ANKEBUT land cover classification task. The combination of low validation loss, high validation pixel accuracy, and visually coherent predicted masks confirms that the model can provide reliable terrain information for the fire risk assessment stage.



Figure 3. 1 Qualitative validation result showing the input RGB image, ground-truth segmentation mask, and predicted segmentation mask.

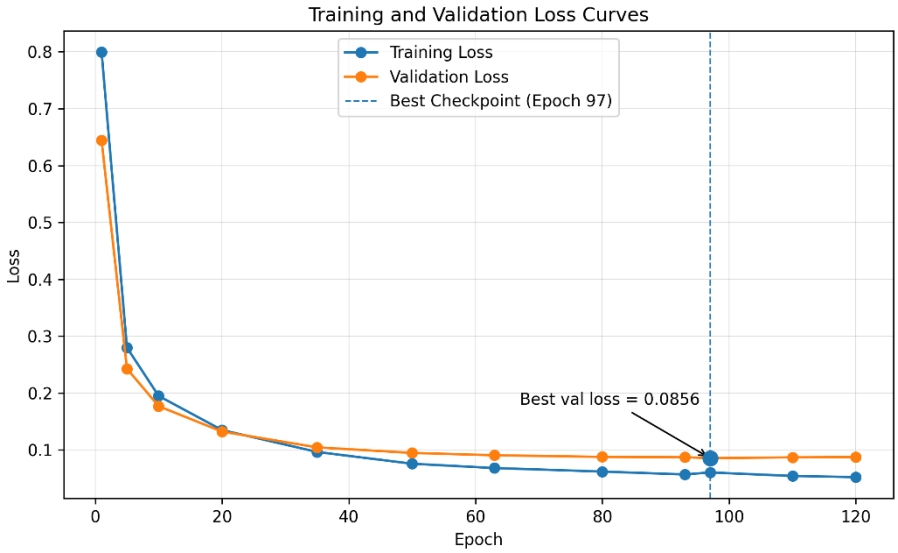


Figure 3. 2 Training and validation loss curves of the DeepLabV3-MobileNetV3-Large segmentation model over 120 epochs.

3.4. YOLOv11 Proxy Detection Results

This section presents the validation results of the YOLOv11 proxy detection model used to test the object detection stage of the ANKEBUT system under safe and controlled conditions. As explained in Chapter 2, real fire and smoke testing was not performed due to safety limitations. Therefore, a proxy detection model was trained using red and green pillows to represent fire- and smoke-related targets during validation.

The proxy YOLOv11 model was evaluated using standard object detection metrics, including precision, recall, mAP@0.5, and mAP@0.5:0.95. Precision measures how many of the predicted detections are correct, while recall measures how many of the actual objects are successfully detected. The mAP@0.5 metric evaluates detection performance at an Intersection over Union threshold of 0.5, while mAP@0.5:0.95 provides a stricter evaluation across multiple IoU thresholds.

Table 0.4 Validation performance of the proxy YOLOv11 detection model.

Metric	*Value*
Precision	0.997
Recall	0.976
mAP@0.5	0.987
mAP@0.5:0.95	0.839

The validation results show that the proxy YOLOv11 model achieved strong detection performance under the controlled testing conditions. The precision value of 0.997 indicates that almost all predicted detections were correct, while the recall value of 0.976 shows that the model successfully detected most of the proxy targets in the validation set. The mAP@0.5 value of 0.987 confirms high detection accuracy at the standard IoU threshold, and the mAP@0.5:0.95 value of 0.839 indicates that the model also maintained good bounding box localization quality under stricter evaluation conditions.

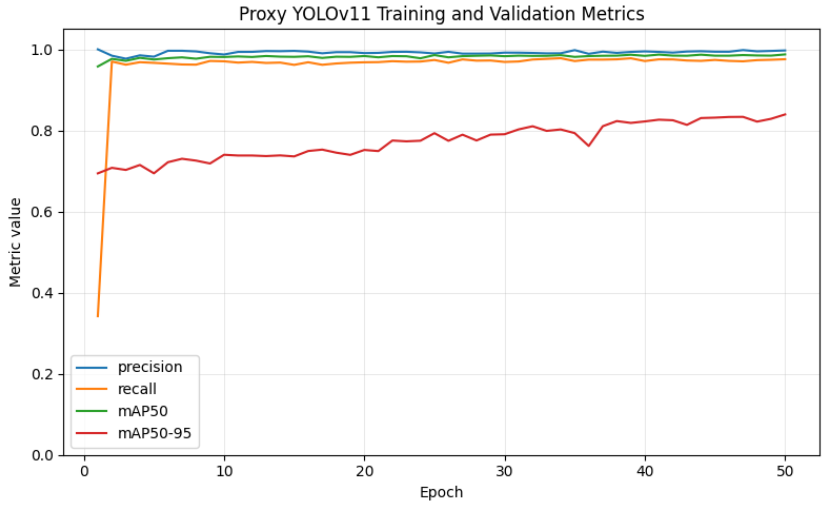


Figure 3. 3 Training and validation metric curves of the proxy YOLOv11 detection model.

Figure 3.3 shows the training and validation metric curves generated from the YOLOv11 results log. The curves demonstrate that the proxy model reached high precision, recall, and mAP@0.5 values during training, while mAP@0.5:0.95 improved more gradually and stabilized near the end of training. This behavior indicates that the model learned the proxy detection task effectively and achieved stable validation performance.

Figure 3.4 presents a qualitative detection example from the trained proxy YOLOv11 model. The figure shows that the model can correctly detect the proxy objects and assign the corresponding class labels with bounding boxes. This confirms that the object detection stage can generate practical outputs such as class label, confidence score, and target location.

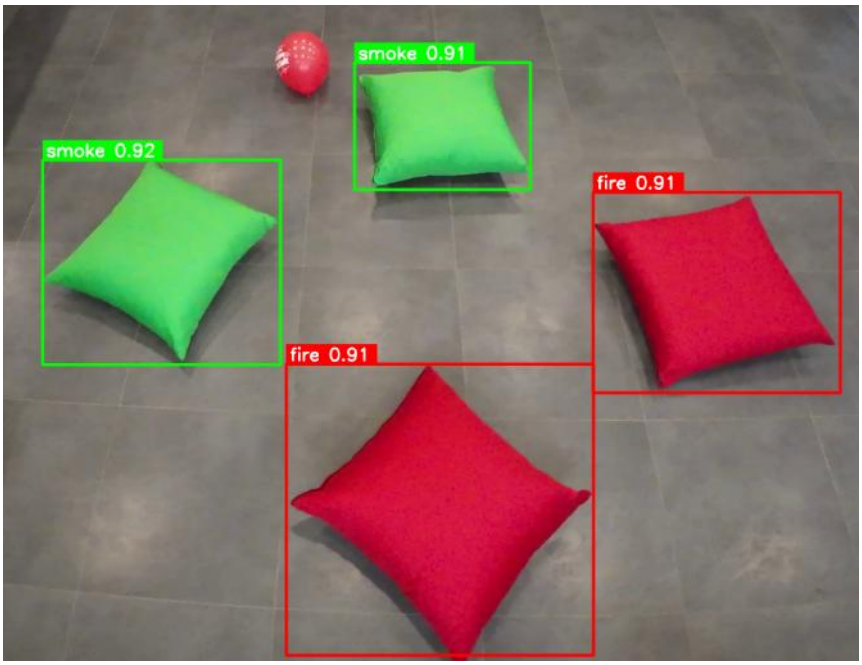


Figure 3. 4 Qualitative validation result of the proxy YOLOv11 detection model showing detected fire and smoke proxy targets with bounding boxes and confidence scores.

The high validation performance of the proxy model demonstrates that the ANKEBUT detection pipeline can operate reliably in a safe testing scenario. Although this proxy model does not replace the production fire and smoke detection model, it validates the complete object detection workflow used by the system. Therefore, the proxy YOLOv11 model was used as the main experimental detection model for confirming that the Jetson Orin Nano can process visual input, run YOLOv11 inference, and provide detection outputs suitable for the downstream alert generation pipeline.

3.5. Wind-Aware Risk Assessment Output

This section presents the results of the wind-aware fire risk assessment stage of the ANKEBUT UAV system. The purpose of this stage is to combine land cover segmentation results, YOLOv11 detection outputs, UAV telemetry, GPS position, and wind direction/speed information to produce directionally aware fire risk warning. Unlike simple fire detection, this approach considers both the combustibility of the observed terrain and the probable direction of fire spread.

The segmentation model provides the base land cover map by classifying each image region into semantic classes such as water, soil, green grass, dry grass, and trees. Each class is associated with a combustibility coefficient. Non-combustible or low-risk areas such as water and soil receive lower values, while highly combustible classes such as dry

grass and trees receive higher values. This produces a spatial combustibility map of the observed area.

Wind data are then used to modify the risk estimation according to the probable spread direction. Wind direction and wind speed information obtained from the TSMS/MGM data source are used to determine the downwind direction. Areas located in the downwind direction from a detected or suspected risk source are assigned increased risk importance because fire spread is more likely to develop in that direction. Therefore, the final output is not only a static combustibility map, but a wind-aware risk estimation that considers both terrain condition and meteorological influence.

During controlled validation scenarios, the risk assessment module successfully combined segmentation, detection, GPS, and wind-related inputs to generate warning outputs. High-combustibility land cover regions such as dry grass and trees produced higher risk values than low-combustibility regions such as water and bare soil. When wind information was included, the generated risk direction was oriented according to the downwind direction, allowing the system to estimate the probable spread tendency.

The results show that the wind-aware risk assessment stage can convert raw perception outputs into meaningful fire risk information. By combining terrain combustibility with wind direction and UAV position, the system provides more informative warnings than a detection-only approach. This supports the main objective of the ANKEBUT system: proactive fire risk awareness before or during early fire development.

Table 0.5 Functional validation summary of the wind-aware risk assessment stage.

Function	*Validation Status*	*Explanation*
Segmentation-based combustibility mapping	Verified	Land cover classes were converted into combustibility-based risk values.
Wind direction integration	Verified	Wind direction was used to estimate the probable downwind spread direction.
Wind speed consideration	Implemented	Wind speed was included as a factor affecting risk severity.
GPS-based alert association	Implemented	UAV position data was used to associate risk outputs with geographic location.
Risk classification	Implemented	Risk outputs were categorized into severity levels such as Moderate, High, and Critical.
Alert communication to GCS	Verified	Generated warning information was transmitted to the ground station during validation.

3.6. Ground Control Station Integration Results

The custom Ground Control Station (GCS) application was validated through integration tests covering Mission Planner embedding, live video reception, MAVLink telemetry

display, and operator-side monitoring. The purpose of this validation was to confirm that the GCS could combine autopilot control, visual feedback, and telemetry information within a single operator interface.

Mission Planner embedding was tested by launching Mission Planner as an independent application and then embedding its window inside the PySide6-based GCS interface. After embedding, the Mission Planner panel remained functional for normal autopilot-related operations, including map interaction, waypoint viewing, mission monitoring, and parameter checking. This confirmed that the embedding procedure did not interfere with the main functionality of Mission Planner.

The video reception pipeline was first validated in a controlled streaming test. A live camera feed was transmitted as an H.264 video stream over UDP and received by the custom GCS video panel. The video widget successfully displayed the incoming stream, confirming that the receiver side of the GCS could support real-time visual monitoring. After the PC-based test, the same video reception workflow was tested with the Jetson Orin Nano acting as the video transmitter. The GCS was able to receive and display the live stream without requiring changes to the receiver interface, confirming compatibility between the Jetson video sender and the ground station software.

MAVLink telemetry reception was also validated using telemetry messages containing flight-related information such as altitude, ground speed, heading, battery voltage, and GPS position. The telemetry panel correctly parsed and displayed the received data in the GCS interface. This provided the operator with a simplified real-time summary of the UAV state in addition to the detailed information available inside Mission Planner.

The integration results also demonstrated the effectiveness of the communication subsystem. The embedded Mission Planner panel, custom live video panel, and telemetry summary panel operated together within the same GCS interface, while the dual-channel communication structure allowed video and telemetry data to remain separately managed during validation tests. This improved operator situational awareness and reduced the risk of losing flight-state information during temporary video degradation.

The Mission Planner embedding test further confirmed that the custom GCS does not replace the original autopilot ground station functionality, but extends it with additional monitoring features. The operator can continue using Mission Planner for mission

supervision while also viewing live video and summarized telemetry in the same application environment.

Overall, the GCS integration results show that the developed interface successfully supports UAV mission monitoring, live visual feedback, telemetry awareness, and operator supervision. This confirms that the ground station component is suitable for supporting the ANKEBUT end-to-end validation workflow.

Figure 3.5 shows the developed Ground Control Station software interface, including the embedded Mission Planner panel, live video panel, and telemetry summary panel. This figure demonstrates the final operator-side interface used to supervise the ANKEBUT UAV system during integration and validation tests.

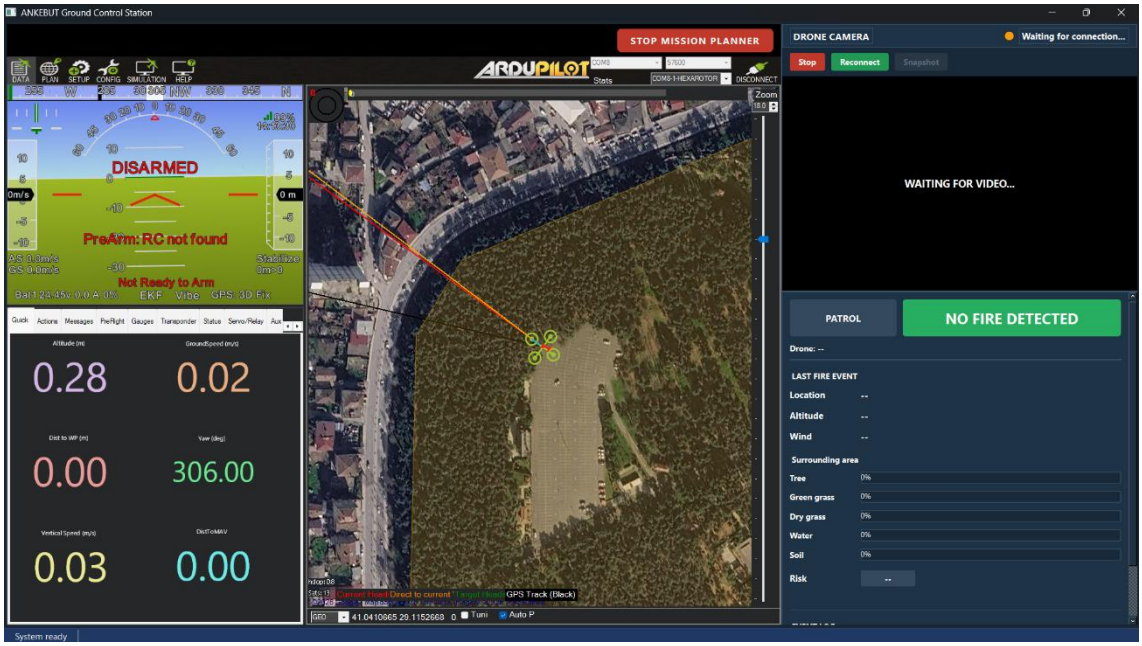


Figure 3.5 Ground Control Station software interface showing embedded Mission Planner, live video display, and telemetry summary panel.

3.7. Autonomous Mission and End-to-End Validation

The end-to-end validation of the ANKEBUT UAV system was conducted to verify that the main hardware, software, perception, communication, and ground station components could operate together as a complete workflow. The purpose of this validation was not only to test individual modules separately, but also to confirm that camera input, onboard processing, detection output, telemetry data, and operator notification could be connected within a single operational pipeline.

During validation, the UAV system was prepared for autonomous grid-based monitoring using a pre-planned mission uploaded through the ground control station. The Cube

Orange flight controller managed the autonomous flight-related tasks, including waypoint navigation, flight stabilization, and landing/RTL behaviors. The Jetson Orin Nano operated as the onboard mission computer and processed the visual data received from the RGB camera.

Because real fire and smoke testing could not be performed safely, the proxy pillow detection model was used during the end-to-end validation process. In this setup, the red pillow represented a fire-related target and the green pillow represented a smoke-related target. This allowed the system to test the detection-to-alert workflow under safe and repeatable conditions without generating real fire or smoke.

The validation workflow followed the complete system chain. First, the RGB camera captured the scene during the test. Then, the Jetson Orin Nano processed the incoming frames and applied the YOLOv11 proxy detection model. When a proxy target was detected, the system produced a detection output including the class label, confidence score, and bounding box location. This output was then used by the alert logic and transmitted to the ground control station for operator awareness.

The ground control station successfully supported operator monitoring during the validation process. The operator could observe the mission state through Mission Planner, view telemetry information, and monitor the live video stream. The successful operation of these components confirmed that the ANKEBUT system could connect autonomous mission execution, onboard perception, communication, and operator-side monitoring in a practical test environment.

Figures 3.6 and 3.7 show the operator-side GCS during the end-to-end validation process, including proxy fire detection, analysis status, UAV telemetry, and mission response.

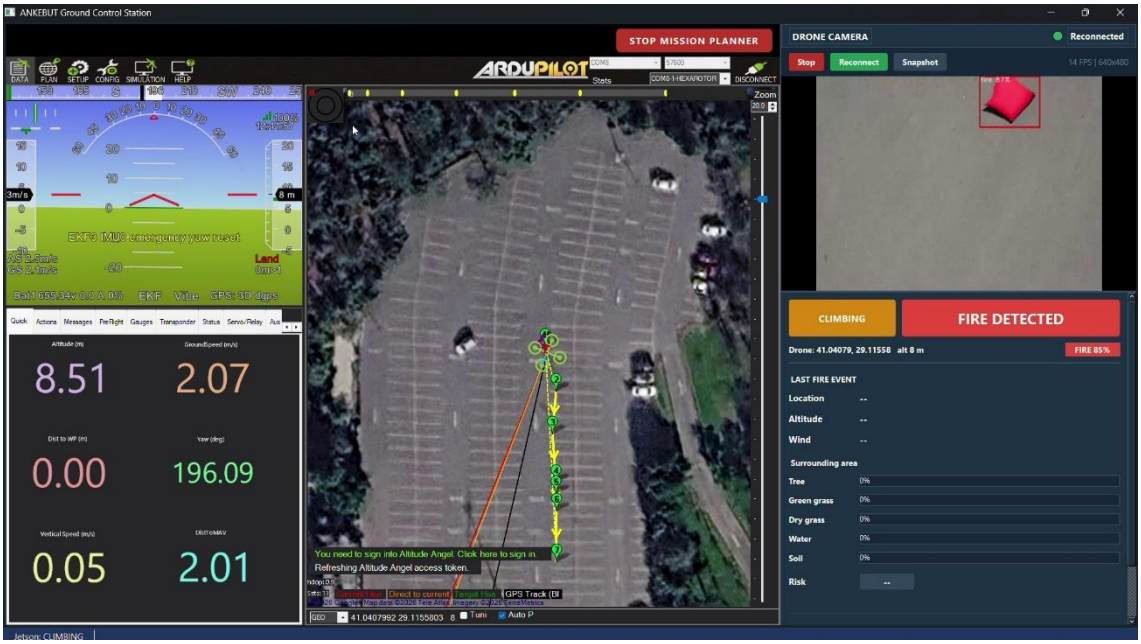


Figure 3.6 GCS interface during proxy fire detection, showing the live detection overlay, fire alert status, UAV telemetry, and climbing response.

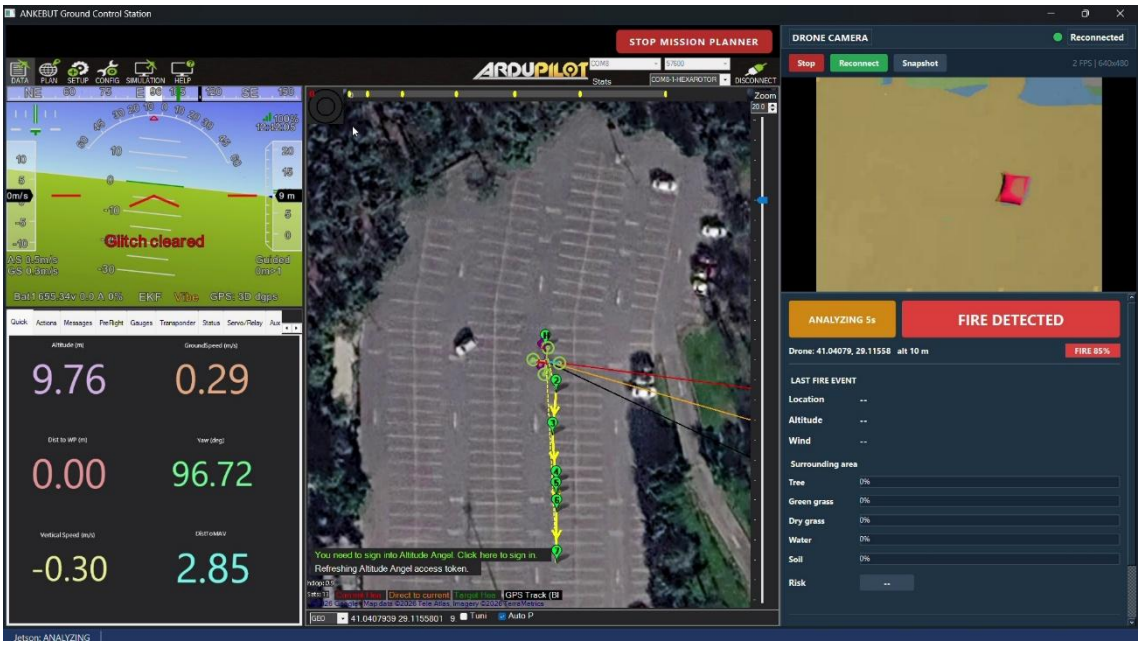


Figure 3.7 GCS interface during the analysis stage after proxy fire detection, showing segmentation-based visual processing, fire alert status, telemetry, and Mission Planner monitoring.

Overall, the end-to-end validation demonstrated that the ANKEBUT UAV system can operate as an integrated aerial monitoring platform. The proxy validation confirmed that the onboard detection module can generate usable outputs and that these outputs can be delivered to the ground station for operator awareness. This result supports the feasibility of the system architecture and provides a safe validation basis for future testing with improved real fire and smoke detection models.

3.8. Discussion

The results obtained from the ANKEBUT UAV system show that the proposed architecture is feasible for proactive forest fire risk assessment using a cost-effective RGB-based perception approach. The system successfully combines autonomous UAV operation, onboard AI inference, wind-aware risk estimation, and ground control station monitoring. Instead of relying only on reactive fire detection, the system uses land cover segmentation to estimate terrain combustibility and integrates wind information to support directional fire spread awareness.

The DeepLabV3-MobileNetV3-Large segmentation model achieved strong validation performance, with a validation pixel accuracy of 96.85%. This result indicates that the model can classify the main land cover categories with high overall accuracy. The segmentation results are especially important because they provide the base combustibility information used by the risk assessment stage. However, some visual similarity between green grass and dry grass may still cause class confusion in transitional regions. This limitation is expected because vegetation color and texture can change depending on lighting, seasonal condition, image quality, and camera altitude.

The YOLOv11 proxy detection model also achieved strong validation results under controlled conditions. The model reached high precision, recall, and mAP values, confirming that the object detection pipeline can reliably detect safe proxy targets. This was important for validating the full detection-to-alert workflow without using real fire or smoke. However, the proxy model does not replace the production fire and smoke detection model. It only confirms that the system pipeline can process visual input, generate detections, and deliver useful outputs to the operator.

The production fire and smoke detection model remains an important future deployment component, but real smoke detection is more challenging than proxy object detection. Smoke has unclear boundaries, variable transparency, irregular shape, and visual similarity to haze or clouds. Therefore, future improvement of the production model should focus on better smoke annotation consistency, additional real fire and smoke data, and possibly segmentation-based smoke detection instead of only bounding box detection.

The hardware and communication results show that the two-layer architecture is suitable for UAV-based AI monitoring. The Cube Orange flight controller handles flight-critical

navigation and safety functions, while the Jetson Orin Nano handles high-level perception and risk assessment. This separation improves system reliability because onboard AI processing does not interfere with low-level flight stabilization. The Ground Control Station further improves operator awareness by combining Mission Planner, live video, telemetry monitoring, and alert information in one interface.

Overall, the ANKEBUT UAV system demonstrates a practical framework for proactive wildfire risk monitoring. The main strength of the system is the combination of terrain combustibility analysis, object detection, wind-aware risk estimation, and real-time operator supervision. The main limitations are the absence of real fire field testing, the need for further improvement of the production fire/smoke detection model, and the limited size of the custom segmentation dataset. Despite these limitations, the obtained results support the feasibility of the proposed RGB-based UAV monitoring approach as a cost-effective alternative to systems that depend mainly on thermal imaging.

4. CONCLUSION AND FUTURE WORK

4.1. Conclusion

This thesis presented the design, implementation, and validation of the ANKEBUT UAV system, an autonomous hexacopter-based platform developed for proactive forest fire risk assessment. The main objective of the system was to move beyond conventional reactive fire detection approaches by analyzing environmental fire risk before or during early fire-risk conditions. Instead of depending only on visible flames, smoke, or expensive thermal imaging sensors, the proposed system uses RGB imagery, semantic segmentation, object detection, meteorological wind data, UAV telemetry, and ground station monitoring to support early fire risk awareness.

The ANKEBUT system was designed as an integrated UAV platform combining flight control, onboard artificial intelligence processing, sensing, wireless communication, and operator supervision. The Cube Orange flight controller was used for flight-critical operations such as stabilization, waypoint navigation, mission execution, and failsafe behavior, while the NVIDIA Jetson Orin Nano companion computer was used for onboard perception and risk-related processing. This separation allowed flight safety and AI processing to be handled by different subsystems, improving the reliability and maintainability of the overall architecture.

A DeepLabV3-MobileNetV3-Large semantic segmentation model was trained to classify aerial RGB images into six land cover classes: background, dry grass, green grass, water, trees, and soil. These classes were selected according to their relevance to fire risk characterization and combustibility estimation. The segmentation model was trained on a custom dataset of 1,170 image-mask pairs and achieved a validation pixel accuracy of 96.85%, with the best validation loss of 0.0856 obtained at epoch 97. These results show that RGB-based semantic segmentation can provide useful terrain information for combustibility mapping and fire risk assessment.

In addition to land cover segmentation, a YOLOv11-based object detection module was developed to provide object-level detection outputs. Since real fire and smoke testing could not be performed safely during validation, a proxy detection model was trained using red and green pillows as safe substitutes for fire and smoke targets. The proxy YOLOv11 model achieved strong validation results and was used to validate the detection-to-alert workflow under controlled conditions. This demonstrated that the

onboard system could process visual input, detect target objects, and provide detection ouputs suitable for the downstream alert generation pipeline.

The wind-aware risk assessment component combined segmentation-based combustibility information with wind direction, wind speed, UAV position, and detection outputs. This allowed the system to estimate not only whether an area is combustible, but also the probable direction in which fire risk may increase under the effect of wind. The Ground Control Station further supported operator awareness by integrating Mission Planner, live video display, telemetry monitoring, and alert-related information into a single interface.

Overall, the results demonstrate that the ANKEBUT UAV system provides a feasible and cost-effective framework for proactive forest fire risk monitoring using RGB-based artificial intelligence and meteorological data integration. Although additional development is required before operational deployment, especially for real fire and smoke detection under uncontrolled field conditions, the implemented system provides a strong foundation for future UAV-based fire risk assessment and early warning applications.

4.2. Main Contributions

The main contributions of this thesis can be summarized as follows:

- *Autonomous UAV-based fire risk monitoring architecture:* An integrated UAV system architecture was developed for proactive forest fire risk assessment. The system combines the Cube Orange flight controller, NVIDIA Jetson Orin Nano companion computer, RGB camera, telemetry communication, wireless video transmission, and custom Ground Control Station software within a single autonomous aerial monitoring platform.
- *RGB-based land cover segmentation for combustibility analysis:* A DeepLabV3-MobileNetV3-Large semantic segmentation model was trained to classify aerial RGB images into six fire-risk-related land cover classes: background, dry grass, green grass, water, trees, and soil. The model achieved 96.85% validation pixel accuracy, demonstrating the feasibility of using RGB imagery for land cover understanding without depending on thermal cameras.
- *Custom dataset preparation and annotation workflow:* A structured dataset preparation pipeline was developed, including image collection, cropping,

CVAT-based manual annotation, data augmentation, mask preprocessing, class index conversion, and dataset splitting. This workflow produced a 1,170-sample segmentation dataset suitable for supervised deep learning training.

- *Combustibility-based risk mapping approach:* Each segmentation class was assigned a combustibility coefficient according to its relative contribution to fire spread potential. This allowed segmentation outputs to be transformed into a risk-aware land cover representation that could be used in the downstream risk assessment pipeline.
- *Wind-aware fire risk estimation:* The system integrated wind direction and wind speed information from TSMS/MGM with segmentation-based combustibility outputs. This enabled the system to estimate directional fire spread risk rather than producing only a static land cover classification result.
- *Safe YOLOv11 proxy detection validation method:* A YOLOv11 proxy detection model was trained using red and green pillows as safe substitutes for fire and smoke targets. This allowed the detection-to-alert workflow to be validated safely without creating real fire or smoke hazards during testing.
- *Custom Ground Control Station integration:* A PySide6-based Ground Control Station interface was developed to combine Mission Planner integration, live video display, telemetry monitoring, and alert awareness. This interface improved operator supervision by presenting key mission information within a single monitoring environment.
- *End-to-end system validation workflow:* The ANKEBUT system was tested as an integrated workflow connecting autonomous mission execution, onboard perception, wind-aware risk estimation, detection output generation, communication, and operator-side monitoring. This confirmed the practical feasibility of the proposed UAV-based fire risk assessment concept.

4.3. Limitations

Although the ANKEBUT UAV system achieved promising results, several limitations should be considered when interpreting the outcomes of this thesis.

- First, real fire and smoke field testing was not performed due to safety and environmental constraints. Instead, the detection-to-alert workflow was validated using a safe proxy model based on red and green pillows. This approach allowed

the system pipeline to be tested safely, but it does not fully represent the complexity of real fire and smoke behavior under outdoor conditions.

- Second, the production fire and smoke YOLOv11 model requires further improvement before real deployment. Smoke detection is particularly challenging because smoke has unclear boundaries, variable transparency, irregular shapes, and visual similarity to haze, clouds, or bright background regions. The annotation strategy for smoke regions also requires additional refinement because inconsistent bounding boxes can negatively affect model performance.
- Third, the land cover segmentation dataset was limited in size and environmental diversity. Although the model achieved strong validation performance, the dataset may not fully represent all possible seasonal, lighting, altitude, and terrain conditions found in real forest environments. Additional images from different regions and flight conditions would improve the robustness of the segmentation model.
- Fourth, the segmentation evaluation was based on validation pixel accuracy and validation loss. A separate held-out test set was not defined, and per-class metrics such as Intersection over Union (IoU), precision, recall, and F1-score were not computed in the current evaluation stage. Therefore, future evaluation should include more detailed class-level performance analysis.
- Fifth, the wind-aware risk assessment module provides a practical risk estimation approach, but it is not a complete physical fire spread simulator. The current system considers land cover combustibility and wind direction/speed, but other important fire behavior factors such as terrain slope, vegetation moisture, air temperature, humidity, and fuel continuity were not fully modeled.
- Finally, communication range and operational endurance remain practical limitations of the current prototype. The UAV endurance is suitable for short validation missions, but larger operational areas would require longer flight time, stronger communication links, or multi-UAV coordination. Similarly, the current wireless video and telemetry setup is appropriate for controlled testing, but operational deployment may require longer-range communication technologies such as 4G/5G or stronger dedicated radio links.

4.4. Future Work

Several future improvements can be made to increase the performance, reliability, and operational readiness of the ANKEBUT UAV system.

- *Expansion of the land cover segmentation dataset:* The segmentation dataset should be expanded with additional aerial images collected under different seasons, lighting conditions, altitudes, camera angles, and geographic regions. This would improve the model's ability to generalize to real forest environments and reduce possible confusion between visually similar classes such as green grass and dry grass.
- *Per-class segmentation evaluation:* Future evaluation should include per-class Intersection over Union (IoU), precision, recall, and F1-score metrics. These metrics would provide a more detailed understanding of how well the segmentation model performs for each land cover class rather than relying only on overall pixel accuracy.
- *Improvement of the production fire and smoke detection model:* The real fire and smoke YOLOv11 model should be improved by collecting more diverse fire and smoke images, refining smoke annotation consistency, and balancing object instances between classes. Since smoke is difficult to represent using simple bounding boxes, future work may also investigate segmentation-based smoke detection.
- *Controlled real fire and smoke validation:* Although proxy pillow testing allowed safe validation of the detection-to-alert workflow, future work should include supervised real fire and smoke experiments under controlled and authorized conditions. Such tests would allow the production detection model and risk warning system to be evaluated under more realistic environmental conditions.
- *More advanced fire spread modeling:* The current risk assessment approach can be extended by including additional environmental factors such as terrain slope, vegetation moisture, temperature, humidity, and fuel continuity. These factors would make the risk estimation more physically realistic and closer to established fire behavior modeling approaches.

- *Communication range improvement:* Future versions of the system may integrate longer-range communication technologies such as 4G/5G, stronger telemetry radios, or adaptive video streaming. This would allow the UAV to operate over larger areas while maintaining stable video, telemetry, and alert communication with the ground station.
- *Ground Control Station enhancement:* The custom GCS can be improved by adding clearer alert visualization, map-based risk overlays, mission history logging, and automatic report generation. These additions would make the system more useful for operators during real monitoring missions.
- *Integration with official early warning systems:* Future work may also focus on integrating the ANKEBUT alert outputs with national forest fire monitoring and emergency response systems such as OGM and AFAD platforms. This would support faster information sharing and improve the operational value of the system.
- *Multi-UAV coordination:* For larger forest areas, future research can investigate cooperative multi-UAV scanning. Multiple UAVs could divide the mission area, share risk information, and improve coverage efficiency during large-scale monitoring operations.

Overall, these improvements would move the ANKEBUT UAV system closer to real-world deployment. The current prototype provides a strong foundation, while future work can focus on expanding datasets, improving real fire and smoke detection, strengthening communication, and validating the system under operational field conditions.

REFERENCES

[1] Andrews, P. L. (2018). The Rothermel surface fire spread model and associated developments: A comprehensive explanation. General Technical Report RMRS-GTR-371. Fort Collins, CO: U.S. Department of Agriculture, Forest Service, Rocky Mountain Research Station.

[2] Chen, L. C., Papandreou, G., Schroff, F., & Adam, H. (2017). Rethinking atrous convolution for semantic image segmentation. arXiv preprint arXiv:1706.05587.

[3] Chen, L. C., Zhu, Y., Papandreou, G., Schroff, F., & Adam, H. (2018). Encoder-decoder with atrous separable convolution for semantic image segmentation. In Proceedings of the European Conference on Computer Vision (ECCV), pp. 801–818. Springer, Cham.

[4] Howard, A., Sandler, M., Chu, G., Chen, L. C., Chen, B., Tan, M., ... & Adam, H. (2019). Searching for MobileNetV3. In Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV), pp. 1314–1324.

[5] Jocher, G., & Qiu, J. (2024). Ultralytics YOLO11 [Software]. Version 8.3.0. Available at https://github.com/ultralytics/ultralytics. ORCID: 0000-0001-5950-6979, 0000-0002-7603-6750.

[6] Merino, L., Caballero, F., Martinez-de-Dios, J. R., Maza, I., & Ollero, A. (2012). An unmanned aircraft system for automatic forest fire monitoring and measurement. Journal of Intelligent & Robotic Systems, 65(1–4), 533–548. https://doi.org/10.1007/s10846-011-9560-x

[7] Rothermel, R. C. (1972). A mathematical model for predicting fire spread in wildland fuels. Research Paper INT-115. Ogden, UT: U.S. Department of Agriculture, Forest Service, Intermountain Forest and Range Experiment Station.

[8] Shamsoshoara, A., Afghah, F., Razi, A., Zheng, L., Fulé, P. Z., & Blasch, E. (2021). Aerial imagery pile burn detection using deep learning: The FLAME dataset. Computer Networks, 193, 108001. https://doi.org/10.1016/j.comnet.2021.108001

[9] Turkish State Meteorological Service (Meteoroloji Genel Müdürlüğü, TSMS / MGM). (2025). Open Meteorological Data API Documentation. Retrieved from https://www.mgm.gov.tr

[10] Yuan, C., Zhang, Y., & Liu, Z. (2015). A survey on technologies for automatic forest fire monitoring, detection, and fighting using unmanned aerial vehicles and remote sensing techniques. Canadian Journal of Forest Research, 45(7), 783–792. https://doi.org/10.1139/cjfr-2014-0347

APPENDIX

Appx. A. COST ANALYSIS AND BUDGET TABLE

No	Material / Service	Qty	Unit Price (TL)	Total (TL)	Description
1	Jetson Orin Nano Developer Kit	1	22.581,51	22.581,51	Onboard AI computer used for segmentation, YOLOv11 inference, video handling, and mission-level processing.
2	Cube Orange Standard Set + Here 3 GPS Combo Package	1	37.777,95	37.777,95	Main flight-control and GNSS subsystem used for stabilization, waypoint navigation, telemetry, and failsafe behavior.
3	SunnySky X4108S 380KV Brushless Drone Motor	6	2.838,00	17.028,00	Propulsion motors used for the S550 hexacopter lift, hovering, and maneuvering.
4	Hobbywing XRotor 40A 2-6S ESC	6	795,50	4.773,00	Electronic speed controllers used to drive and regulate the six brushless motors.
5	Holybro SiK Telemetry Radio Module V3, 433 MHz	1	5.558,00	5.558,00	Long-range MAVLink telemetry link between the UAV and the ground control station.
6	Pixhawk/APM Compatible 30V 90A Power Module	1	328,77	328,77	Power monitoring and regulated power supply interface for the flight-controller system.
7	Alfa Network AWUS036ACH 802.11ac Wireless USB Adapter	2	6.500,00	13.000,00	5 GHz wireless communication link used for video and data communication between Jetson and the GCS.
8	22.2V 6S 6200mAh 45C LiPo Battery	1	6.290,47	6.290,47	Main propulsion battery used to power the motors and primary UAV flight system.
9	3S 1800 mAh LiPo Battery	1	2.340,05	2.340,05	Separate battery used to power the Jetson Orin Nano independently from the main propulsion battery.
10	Propeller Set (CW & CCW)	4	71,53	286,12	Propeller sets used with the hexacopter propulsion system for thrust generation.
11	S550 Carbon Fiber Drone Frame	1	3.291,60	3.291,60	Main hexacopter airframe used for mounting motors, avionics, battery, camera, and communication hardware.
			TOTAL	**113.255,47**	

Table A1 Cost analysis and budget breakdown of the ANKEBUT UAV project.